

Onyx — Kernel internals

Onyx · multi-process OS on Raspberry Pi 4

This document describes the inner workings of the **Onyx kernel** (the code under `kernel/`). It does **not** describe Circle, which is used as the HAL/driver stack; we only point out the seams with Circle where necessary.

All file paths are relative to the repository root. The values (addresses, constants) come from the code; the layout constants live in `kernel/include/kern/layout.h`.

Contents

1. Scope: ours vs. Circle
2. Boot sequence
3. Address-space map
4. Memory management: `CAddressSpace`
5. Scheduling
6. Exceptions and vectors
7. ELF loader and process model
8. The kapi ABI table
9. Stream / stdio subsystem
10. Graphics subsystem (GUI)
11. Network subsystem (WLAN, TCP/IP, NTP)
12. Sound and the second core
13. Post-mortem debug console
14. App cores (cores 2 and 3)

1. Scope: ours vs. Circle

Written by Onyx	Reused from Circle
Per-process address spaces (<code>mm/addrspace.cpp</code>)	64 KB page allocator (<code>palloc/pfree</code>)
Scheduler (<code>sched/scheduler.cpp</code> , <code>sched/task.cpp</code>)	<code>taskswitch.S</code> , <code>mutex/semaphore/event</code>
VBAR_EL1 vectors, trap frame (<code>arch/aarch64/</code>)	<code>InterruptHandler</code> (GIC + EOI), <code>FIQStub</code>
ELF loader (<code>proc/elf.cpp</code>)	EMMC + FatFs, USB HID
ABI table + kapi impl. (<code>sys/</code>)	<code>C2DGraphics</code> (framebuffer), <code>CTimer</code>
GUI: rendering, compositor, widgets, skins, dialogs (<code>gui/</code>)	<code>CCharGenerator</code> (bitmap font)

The kernel is linked against Circle's static libraries. **Two Circle files are replaced** (`lib/sched/scheduler.cpp` and `task.cpp`): because our `.o` files appear in `OBJS` before the `--start-group`

archives, the linker resolves `CScheduler/CTask` from our versions and never pulls in those from `libsched.a`.

A **shadow header** (`kernel/compat/circle/sched/scheduler.h`) is placed **before** `circle/include` on the include path, so that `<circle/sched/scheduler.h>` resolves to ours (which adds `OnTimerTick()`, the notion of a time slice, etc., while keeping Circle's public + friend API).

2. Boot sequence

The entry point is `main()` (`kernel/main.cpp`), called by Circle's `sysinit` **after** it has already: switched the CPU to EL1, enabled the MMU (64 KB granule), initialized the GIC, run the static C++ constructors.

All the logic lives in the `CKernel` class (`kernel/kernel.cpp`):

`CKernel::Initialize()` — in order

1. **Text console:** `m_Screen` (HDMI) then `m_Serial` (115200). The boot log goes to the HDMI screen so it is visible **without a serial cable**. A `CLogSwitch` routes the logger's output (see §11).
2. **Interrupts + timer:** `m_Interrupt.Initialize()`, `m_Timer.Initialize()`.
3. **Exception vectors:** `install_vectors()` installs our `VBAR_EL1` (`KVectorTable`) over Circle's.
4. **Scheduler:**
 - `m_Timer.RegisterPeriodicHandler(PeriodicTick)` — wires the tick to `OnTimerTick()`.
 - `AddrSpaceInit()` — captures the kernel's TTBR0 (base of the shared L2 table).
 - `RegisterTaskSwitchHandler(AddressSpaceTaskSwitch)` — on each task switch, activates the new task's address space.
 - `RegisterTaskTerminationHandler(AddressSpaceTaskTerminate)` — frees the address space when a task ends.
5. **ABI table:** `KApiTableInit()` fills the `kapi` pointer table (see §8).
6. **Graphics + SD:** `m_2DGraphics.Initialize()` (32 bpp framebuffer); `m_EMMC + f_mount()` (FatFs) of SD: (partition 1, the boot FAT32 one), then of **SD1: ... SD3:** (partitions 2–4) when they hold a FAT or **exFAT** file system (see *Volumes* in §8). Loading the skins from `SD:skins/` (`wings.bmp`, the cursor `mousecur.bin`, `theme.txt`).
7. **USB:** `m_USB.Initialize()` (mouse + HID keyboard, hot-plug).

`CKernel::Run()` — the service loop

1. Logs machine info.
2. Launches the **init program** (`StartAutostart`): the ELF named by the `cmdline.txt` option `init=` (e.g. `init=SD:/bin/init`), defaulting to `SD:bin/init` when the option is absent. The kernel does not parse any launch list itself — `init` (PID-1 style) reads `SD:/etc/autostart` and starts everything from there (by default `voronoy`, which paints the wallpaper then exits, and `panel`, the shell). Pointing `init=` at a different ELF swaps the whole userland launcher (e.g. a recovery shell) without rebuilding the kernel.
3. Launches the **kernel service tasks**:
 - `CCompositorTask` — presents the screen at up to ~60 Hz, **only what changed** (dirty rectangles): every visual change says where — `ScreenDirtyRect` for a part (`kapi_present/SYS_present` and `chrome`: the caller's window; window add / remove / raise / move / resize / alpha: its old and new places; the cursor: its old and new 16×16

spots, with the drag badge) or `ScreenDirty` for all of it (wallpaper, full screen end, drag & drop start / end). Rectangles that touch are merged (at most 16; more, or over 2/3 of the screen, = the whole screen) and `g_nScreenGen` is bumped. The task takes the damage (`ScreenTakeDamage`) and for each rectangle composites with the screen image **clipped** to it (`GImage::SetClip`: every blit and fill stays inside) and sends just that rectangle to the display (`C2DGraphics::UpdateDisplay (x, y, w, h)`), our Circle patches 6 / 12: the frame buffer's 2D DMA reads the rectangle in place, source and destination strides; `CCompositorTask::Present` starts it with `UpdateDisplayAsync` and **yields** to the other tasks until its interrupt says it is done, instead of spinning); the whole screen only when told, plus a safety refresh every 2 s. A window animating (a game, an emulator) costs its own area instead of the whole screen, a mouse move two cursor-sized spots. An idle desktop thus costs almost no CPU (the heartbeat's fps shows the real composites). It is created **before** `init` (no boot-log pause any more) and the main task hands it the CPU at once with `CScheduler::YieldTo()` (scan starts at that task instead of the round-robin next), so it presents before the first app's window. Starting it late — after a 6 s pause, with a single GUI app already running — used to hang the boot.

- **CReaperTask** — reaps terminated tasks every ~50 ms (frees `CTask` + stack + address space outside the scheduler core). It also blinks the green ACT LED as a headless sign of life: 1 s period while the network is down, 0.2 s once `NetIsUp()`; a frozen LED means the scheduler stopped.
- **CInputTask** (if USB is present) — pumps keyboard/mouse events to the window manager; every 100 ms it also looks for new gamepads (`upad1..4`) and USB MIDI devices (`umidi1..4`, v68: §8 *USB MIDI*), also after a hot plug.
- **CGuiWatchdogTask** (skipped with `watchdog=0` in `cmdline.txt`) — once a second, checks the GUI and writes to the kernel log (`kmsg`): `compositor STALLED` when `CWindowManager::FrameCount()` has not moved for 2 s (with every task's `name:state`), app '`<title>`' **NOT PUMPING** events when a window has queued events and its owner has not called `PopEvent` for 2 s (plus the matching "recovered" lines), and a **heartbeat** every `heartbeat=` seconds from `cmdline.txt` (default 5, 0 = off): uptime, fps, mouse/key deltas, tasks by state, windows with queued/dropped events. Counters live in `CWindowManager` (frames, `OnMouse/OnKey` calls, `Snapshot()`) and `CWindow` (`QueuedEvents`, `DroppedEvents`, `LastPumpTicks`).

4. The main task runs an idle loop (`MsSleep`).

3. Address-space map

One `TTBR0` per process. The kernel (identity) must stay at the **low VAs** (because identity requires `VA == PA`), so **kernel and user coexist in `TTBR0`**: the kernel L2 entries are **shared**, the user L2/L3 are **per process**.

VA	Content	Attributes
0x0_0000_0000	KERNEL IDENTITY (RAM, MMIO, PCIe) mapped by Circle	EL1 RW, global (<code>nG=0</code>), shared in every table (4 GB = <code>KERNEL_IDENTITY_END</code>)
0x1_0000_0000 (4 GB)		
0x2_0000_0000 (8 GB)	unmapped hole (guard)	USER_VA_BASE (8 GB) EL1, ASID-tagged (<code>nG=1</code>) USER_WINDOW_CANVAS (12 GB)
0x3_0000_0000	.text / .data / .bss / heap (app) window canvas	

0x3_4000_0000	wallpaper buffer	USER_WALLPAPER_CANVAS (13 GB)
0x3_8000_0000	kapi ABI table (read-only)	KAPI_TABLE_VA (14 GB)
0x3_9000_0000	generated code (a JIT, RWX EL1)	USER_CODE_BASE (14.25 GB, v58)
0x4_0000_0000	user stack (grows down)	USER_STACK_TOP (16 GB)
(16 GB)		1 MB initial
...		
0x10_0000_0000		T0SZ ceiling (64 GB) on RPi 4

- **64 KB granule, EL1 stage-1 only**, TTBR1 disabled.
- 2 levels: one **L2 entry = 512 MB** (points to an L3 table), one **L3 page = 64 KB**.
- **8-bit ASID** (256 contexts), taken from TTBR0_EL1[63:48].
- USER_LOAD_BASE = USER_VA_BASE: apps are linked at 8 GB (see user.ld).

4. Memory management: CAddressSpace

Source: kernel/mm/addrspace.cpp, kernel/include/kern/addrspace.h.

A CAddressSpace object = a process. It owns: the L2 table, the ASID, a PID, the window pointer, the stdin/stdout streams, the process handle, the exit code, and the argv/cwd string.

Construction

1. **Allocates a fresh L2 table** (palloc, one 64 KB page).
2. **Copies the kernel L2 descriptors** from the template captured at boot (memcpy of the whole L2 page). The kernel entries point to the **same shared L3 tables**; the L2 slots covering the user area stay invalid (zero).
3. **Allocates an ASID** (1..255; 0 is reserved for the kernel/global).
4. **Maps the kapi ABI table** read-only at KAPI_TABLE_VA (14 GB) with KPAGE_ATTR_APP_RODATA. The physical page is a kernel global (not "owned" by this process → never freed at destruction).

MapPage(VA, PA, attrs, bOwned)

Maps a 64 KB page (VA and PA aligned to 64 KB):

- GetOrCreateL3(L2_INDEX(VA)): if the L2 entry is invalid, allocates a fresh L3 table and writes the L2 table descriptor.
- Fills the L3 page descriptor: AttrIdx, AP, SH, AF=1, nG, output = PA, PXN, UXN. The software bit PAGE_SW_OWNED (the Ignored field) marks the frames to free at destruction.

Page attribute matrix (layout.h presets)

Use	AttrIdx	AP	nG	PXN	UXN
KPAGE_ATTR_APP_CODE (app code)	NORMAL	RO_EL1	1	0 (exec. EL1)	1
KPAGE_ATTR_APP_DATA (data/stack /canvas)	NORMAL	RW_EL1	1	1	1
KPAGE_ATTR_APP_RODATA (kapi	NORMAL	RO_EL1	1	1	1

Use	AttrIdx	AP	nG	PXN	UXN
table)					
KPAGE_ATTR_APP_RWX (generated code, v58 code_alloc)	NORMAL	RW_EL1	1	0 (exec. EL1)	1
KPAGE_ATTR_USER_* (ELO legacy, dormant)	NORMAL	*_ALL	1	...	...

App pages are **accessible at EL1** (AP=*_EL1), executable at EL1 for code (PXN=0), never executable at EL0 (UXN=1), and nG=1 (ASID-tagged) → isolation between applications.

Activation and context switch

```
// CAddressSpace::Activate()
u64 ttbr0 = MAKE_TTBR0(phys(L2), asid); // = phys | (asid << 48)
asm volatile ("msr ttbr0_el1, %0; isb" :: "r"(ttbr0) : "memory");
```

No TLB flush on switch (TLB tagged by ASID). The AddressSpaceTaskSwitch hook calls `Activate()` for app tasks, or `ActivateKernelAddressSpace()` for kernel tasks (which have no address space).

Destruction (and an important pitfall)

When destroying the address space:

1. Signals EOF on stdout, releases the streams, marks the process handle as terminated.
2. **Invalidates the TLB for this ASID** before freeing anything: `tlbi aside1is, Xt ; dsb ish ; isb`. (Reusing a page-table frame while the walk is still cached would break translations.)
3. Removes and destroys the window.
4. Walks the **user** L2/L3 slots and frees the frames marked `PAGE_SW_OWNED`.

⚠ **Pitfall: never free an L3 table shared with the kernel.** Because each address space **copies** the kernel L2 descriptors, some L2 entries point to the **same** L3 tables as the kernel. Before freeing an L3 from the user area, the destructor **compares** the table address against the reference kernel L2's and **skips** any shared table. Freeing a shared kernel table would corrupt the translations of the *entire system*. (See also the design note "per-AS L2 copies the kernel L2".)

5. Scheduling

Source: `kernel/sched/scheduler.cpp`, `kernel/sched/task.cpp`,
`kernel/compat/circle/sched/scheduler.h`.

Preemptive scheduling (track A)

The kernel **preempts application code**. A 100 Hz timer tick drives `OnTimerTick`, which counts down the running task's time slice (`SCHED_SLICE_TICKS` = 20 ms) and sets a reschedule flag when it expires.

`KernelIRQExit()` — run at the end of every IRQ — then forces a switch, **but only when it is safe**: the interrupted context must be an **application running its own code**, i.e. `SPSR_EL1.M == EL1t` and the return PC (`ELR_EL1`) lies in the **user VA** range (`IS_USER_VA`). An app inside a `kapi_*` call has a *kernel*-VA return PC and is **not** preempted — that PC test is the kernel lock (the call may hold a kernel resource; such `kapis` yield cooperatively anyway). Kernel threads (`EL1h`, or `EL1t` with a kernel-VA PC) are never preempted.

The SP_EL0/SP_EL1 problem and the trampoline. Circle runs all tasks in `EL1t` (on `SP_EL0`), while the IRQ handler runs in `EL1h` (on `SP_EL1`). A `TaskSwitch` straight from the IRQ would swap `SP_EL1`, not the thread's `SP_EL0` → it cannot preempt an `EL1t` thread. So instead of switching inside the IRQ, `KernelIRQExit` **redirects the return**: it stashes the app's `ELR_EL1/SPSR_EL1` (in `g_PreemptELR/g_PreemptSPSR`), points `ELR_EL1` at `PreemptTrampoline` and masks `IRQ+FIQ`. The IRQ's `RESTORE_TRAP_ERET` then `erets` into the trampoline **at `EL1t`, on the app's own stack**, where it saves the app's full register/FP state and calls the ordinary cooperative `Yield()` — which now swaps the correct `SP_EL0` and address space. When the task is rescheduled, the trampoline restores the app state and `erets` back to the exact interrupted instruction (original `NZCV` + interrupts restored). A preempted task is thus parked **exactly like a voluntary yielder** — one resume path. (The app runs on its `CTask` kernel stack, which is in the global identity region, so it stays mapped across the address-space switch in `Yield`.)

Because only user-VA application code is preempted, the **kernel itself is non-preemptive** (the classic Unix model): a long, non-yielding *kernel* loop would still block other tasks — drop a `if (IsReschedPending()) Yield();` cooperative-preemption point into any such loop if one appears. Tasks also still switch **voluntarily** (`Yield`, `MsSleep`, `present`, `wait`, ...), unchanged.

Preemption points in file I/O. The long kernel operations an app can ask for are cut into pieces with a `Yield` between them: `kapi_read / kapi_save_file` move at most 64 KB per `f_read / f_write` (`ChunkedRead / ChunkedWrite` in `sys/kapi.cpp`), the ELF loader 128 KB. Without them one read of a 28 MB file (a Doom WAD, a GBA ROM, a newlib app's `fopen`, which reads the whole file) stopped every other task — the compositor, the cursor, the sound feeders — for seconds. Between two pieces the FatFs volume lock is free, so other tasks' file calls get through; the caller's buffer stays valid (its address space is active again when it resumes).

File-system speed. The FatFs disk layer keeps a 1 MB **sector cache** of the single-sector reads (the FAT, the folders), write-through (`docs/05 §8`; `sdcache=0` turns it off): opening files and walking folders stopped asking the card for every sector. And `f_read / f_write` move a file's contiguous clusters in **one** SD command (`docs/05 §10`), instead of one per cluster (with 512-byte clusters, one per 512 bytes: 2 MB/s). A read of 1 MB or more logs its time split (`fs: read ... SD: N commands, waiting, data port, other tasks`). The card runs in High Speed (50 MHz, `docs/05 §9`; `sdhs=0` = 25 MHz). The boot log says what is in use (`SD card mounted (SD): ...`); `/bin/fsbench` measures.

Yielding SD waits. The SD driver also yields while the card keeps it waiting: after a 2 ms spin (a normal command ends within that: yielding for it cost a whole time slice per command, 2 MB/s), `CEMMCDevice::TimeoutWait` calls `OnyxDriverWait ()` (a weak hook of our Circle fork, `docs/05 §7`), which `Yields` if IRQs are on. The FatFs volume lock is held during that wait, so it is a **sleeping** lock (`OnyxFsLockTake / OnyxFsLockGive`, `sys/fslock.cpp`, re-entrant, its waiters `Yield`): with Circle's spin lock a second task entering FatFs would spin forever against a holder it never lets run. The holder is in a **no-kill section** (`CScheduler::EnterNoKill / LeaveNoKill`): `TerminateTask` on it only marks it, and it ends in `LeaveNoKill`, once the lock is free — a dead owner would lock the card forever. (Before: 100–200 ms freezes each time the menu bar read the `app.txt` files, or an app opened a file in a big directory.)

Stall watchdog. To find the kernel code that still keeps the CPU too long, the scheduler notes the time of every `Yield()`. When the running task (not idle) has not passed through `Yield()` for more than `SCHED_STALL_US` (100 ms), the IRQ exit path (`KernelIRQExit` → `CScheduler::StallSample`) samples the interrupted PC and LR every `SCHED_STALL_SAMPLE_US` (50 ms, up to 8 samples). The task's next `Yield()` closes the report (task name, duration) into a small ring, and the reaper task writes it to the kernel log:

```
stall: doom ran 850 ms without yielding; pc/lr: 1a2b40/1a2a10 1a2b48/1a2a10 ...
```

Kernel addresses resolve with `kernel/kernel18-rpi4.map` (or `aarch64-none-elf-addr2line -f -e kernel/kernel18-rpi4.elf <pc>`). No sample at all ("IRQs masked") means the task ran with interrupts off the whole time. (Its first catch: `12dbe4/138044` = `LeaveCritical` from `CTerminalDevice::Write` — the boot-console log writes, see §11 `MuteNormal`.)

CPU hogs vs yielders (dynamic priority). Circle's network and Wi-Fi code waits in `Yield()` loops (`qlock/sleep/tsleep` in `addon/wlan/p9proc.cpp`, the `CNetTask Process` loop, socket send/recv — also inside an app's `kapi_*` call, e.g. `vncd`). In plain round-robin, each of those yields hands the CPU to a CPU-bound app for a full slice, so the loops crawl. Driver timeouts then expire and the Wi-Fi link drops (the `spin` test app took the network down, and VNC/telnet with it). So the scheduler lowers the priority of the tasks the timer has to preempt:

- `PreemptDoYield` calls `CScheduler::OnPreempt()`, which bumps the task's **preemption streak** (`m_nPreemptStreak[slot]`). Any voluntary `Yield` (a `kapi` wait, `msleep`, `present`, a kernel task's loop) resets it.
- A task with a streak $\geq$ `SCHED_HOG_STREAK` (2) is a **CPU hog**. An app that computes more than a slice now and then but yields in between (`vncd` encoding a frame, then sending it) is not one.
- A hog's preemption opens a **burst** of `SCHED_BURST_US` (60 ms). During the burst `GetNextTask` **skips the hogs** (and idle): kernel tasks and apps that yield are served first. The burst ends early once none of them is ready.
- A hog also runs with a **one-tick slice** ($\leq$ 10 ms instead of `SCHED_SLICE_TICKS`), so the kernel's yield loops never wait more than $\sim$ 10 ms behind it.

A hog thus keeps roughly 15 % of the CPU while others need it, and all of it otherwise. Both are tunable at boot for A/B testing: `cmdline.txt slice=` (ticks) and `hogsched=0` (plain round-robin) → `CScheduler::Configure`.

CScheduler (shadow) and CTask

- Circle's public + friend API is preserved (`Yield`, `Sleep/MsSleep`, `GetCurrentTask`, `AddTask`, `BlockTask`, `WakeTasks...`) → Circle drivers that block (USB, etc.) work without modification.
- **The task list has no limit.** Circle's table of `MAX_TASKS` (40) pointers — and its `LogPanic` ("System limit of tasks exceeded") when full — is gone: the tasks are a **circular linked list** of `TSchedNodes` (`scheduler.h`), each with the scheduler's own state about its task (the no-kill depth, a pending kill, the preemption streak, killed from outside). `CTask` itself cannot grow (its layout is Circle's, shared with the prebuilt libraries), hence the separate node; `AddTask` allocates it, the reaper frees it. The round robin scans the circle from the node after `m_pScan` (the last one chosen; `YieldTo` points it just before its task); `m_pCurNode` is the running task's node, so the current task's state is reached without a search. `EnumerateTasks`'s callback must not yield.
- **A process's tasks are one group** — the tasks whose `TASK_USER_DATA_USER` is the same address space (the app's threads, §7). `TerminateTask` on an app's task terminates the whole group (`TerminateGroup`: all but the caller); `CountGroup` counts its live ones. A task **killed** from outside

may still be on a wait list of its process (a mutex, an event, a socket): the reaper frees it only once no task of its group is alive, and runs the termination handlers of a batch **before** deleting any of its tasks — the last handler frees the process (its sockets closed, its objects' wait lists emptied) while the tasks still exist.

- `WakeTasks` **tolerates a task that is no longer blocked**: a `BlockTask` with a timeout that expired is made Ready by `GetNextTask` but stays on the event's list until it runs again and unlinks itself; an event set in that window (the V3D's frame-done interrupt right after one of the drawing task's 2 ms waits ended) found it Ready, and Circle's assertion halted all four cores — the random freeze of `n64emu` with the GPU (found with the crash record, `sched/scheduler.cpp(742)`). Such a task is now only unlinked.
- **"Real time" tasks (v68, `SetPriority`, `kapi_thread_priority`)**. A node's `nPrio` (`SCHED_PRIO_HIGH`) makes `GetNextTask` try `ScanPrio` first — only while such tasks exist (`m_nPrioTasks`): the first ready real-time task after `m_pScan` wins, before the burst and the round robin. The current task is left out of that scan, so a real-time thread that yields while still ready (a spin on `kapi_lock` held by an ordinary thread) lets the others run. A tick (`OnTimerTick`, after `WordWaitTick`) that finds one ready while an ordinary task runs sets the resched flag (`m_bPrioPreempt`): the IRQ exit preempts the app at once (in its own code only, as always), and `OnPreempt` does not count that towards the app's hog streak. The priority holds only while the task yields by itself: preempted at the end of its slice (a busy loop), it is marked `bPrioSpent` — an ordinary task — until its next voluntary `Yield` (sleep, wait). So a real-time thread cannot starve the system; an audio pump that renders and sleeps keeps it.
- `Yield()` performs the context switch **with the IRQ disabled** for atomicity, then **restores the full DAIF** of the resumed task (each task keeps its own IRQ enable state).
- The **only** change in `task.cpp: TaskEntry()` re-enables the IRQ (`msr daifclr, #2`) so that a **fresh task** does not start with the IRQ masked inherited from the switcher.
- The idle task uses `wfi` (on core 0; on the network core it only pauses: no timer tick there).
- **One scheduler per core that runs tasks**. `CScheduler::Get ()` returns the scheduler of the calling core (`s_pThis[core]`, the core read from `MPIDR_EL1`): core 0's, and with `netcore=1` core 3's (§11). A `CTask` joins the scheduler of the core that creates it, so Circle's code — tasks, `CSynchronizationEvent`, `MsSleep`, `Yield` — works unchanged on either core. The wait lists of the events share one spin lock (an event may be set from another core). Cores 1-2 have none (`IsActive ()` is FALSE there; `Get ()` falls back to core 0's, as before). Circle's own libraries, built with Circle's `scheduler.h`, read `s_pThis` as a single pointer in their inline `IsActive ()`: element 0, core 0's.

Per-task data

`CTask`'s `TASK_USER_DATA_USER` slot holds the task's `CAddressSpace*` pointer (0 for kernel tasks) — the same one for all the tasks of a process (its threads). The PID, the state (R/S/B/N), the name and the ASID are accessible via the address space and the enumeration API (see §8). The address space counts its tasks (`AddTask` / `DropTask`; the first is its **main task**, `GetMainTask`); the reaping (**reaper**) calls the termination handler — which drops the task, and frees the address space with the last one — then destroys the `CTask`.

6. Exceptions and vectors

Source: `kernel/arch/aarch64/vectors.S`, `kernel/arch/aarch64/exception.cpp`,
`kernel/include/kern/trapframe.h`.

KVectorTable (OUR VBAR_EL1)

16 entries (4 groups × 4 types). A crucial point from hardware bring-up: the **EL1t group (Current EL, SPO)** is the one actually taken by the main thread and the tasks (which run in EL1t). It must route to the real handlers (`SyncEL1Entry` / `IrqEntry` / `FIQStub`) and **not** to `BadModeEntry` — otherwise the first tick crashes.

Group	Sync	IRQ	FIQ	SError
Current EL SPO (EL1t)	SyncEL1	Irq	FIQStub	BadMode
Current EL SPx (EL1h)	SyncEL1	Irq	FIQStub	BadMode
Lower EL AArch64 (EL0)	SyncEL0	Irq	FIQStub	BadMode
Lower EL AArch32	BadMode ×4			

IRQ path

`IrqEntry`: `SAVE_TRAP` → **unmasks the FIQ** (`DAIFClr, #1`, required by Circle's `EnterCritical`) → calls Circle's `InterruptHandler` (GIC dispatch + periodic tick + EOI) → `KernelIRQExit` (**preemptive reschedule** via `PreemptTrampoline` — see §5) → re-masks the FIQ → `RESTORE_TRAP` + `ERET`.

On cores 2 and 3 (app cores, §14) the same `KVectorTable` is installed: their only IRQ is the stop IPI, and `KernelIRQExit` hands it to `AppCoreOnIRQExit`; a synchronous fault there goes to `AppCoreOnFault` instead of the kernel panic.

Bring-up note: leaving the FIQ masked during `EnterCritical` caused a Circle assertion to fail (`synchronize64.cpp`). Hence the `DAIFClr, #1/DAIFSet, #1` around the call, as Circle's `IRQStub` does.

Trap frame

`TTrapFrame` (800 bytes): `x[0..30]`, `sp_el0`, `elr_el1`, `spsr_el1`, then the full FP/SIMD state — `v[0..31]` (the 128-bit q registers), `fpsr`, `fpcr`. Saved by the assembler macro `SAVE_TRAP` onto the stack on exception entry, restored by `RESTORE_TRAP_ERET`.

The FP/SIMD block is saved on **every** trap so floating-point user code keeps its registers across a context switch. The cooperative `TaskSwitch` (Circle) only preserves the callee-saved `d8-d15` mandated by the AArch64 PCS — enough for a voluntary `Yield()`, but **not** enough when the timer preempts a thread mid-computation. Saving the whole register file here (and in `PreemptTrampoline`, see § "Preemptive scheduling") keeps hardware float correct under both voluntary switches and preemption from the IRQ-exit path. FP/SIMD is enabled at EL0/EL1 at boot (`CPACR_EL1`, `circle/lib/startup64.S`), so the `stp/ldp q-` register forms never trap.

System calls (dormant)

The `ESR_EL1.EC` decode recognizes `SVC64 = 0x15`. `SyscallEntry` dispatches on `x8` (number) with args `x0-x5` and return in `x0` (Linux-style ABI). **But** because the apps run in EL1 and call `kapi_*` directly, **this path is not taken in normal operation**; it is only self-tested. The `copy_from_user/ copy_to_user` helpers use the unprivileged `LDTR/STTR` accesses. Synchronous non-SVC EL1 faults trigger a panic dump to the screen (`PanicToScreen` + Circle's register dump).

7. ELF loader and process model

Source: `kernel/proc/elf.cpp`, `kernel/include/kern/elf.h`, task model in `kernel.cpp`.

`LoadELF(image, size, AS, &entry)`

- Validates the ELF64 header (magic, `ELFCLASS64`, `EM_AARCH64=183`, type `ET_EXEC/ET_DYN`).
- For each **PT_LOAD** segment: validates the bounds (file + VA within the user area), chooses the attributes according to `PF_X` (code = `APP_CODE` RO+X, data = `APP_DATA` RW), then `LoadSegment`:
 - maps all the 64 KB pages covering `[vaddr, vaddr+memsz)` (via `MapNewPage`),
 - copies `filesz` bytes from the file (the BSS beyond stays zero).
- `SyncDataAndInstructionCache()` after writing the code, then returns `e_entry`.

`CUserProcessTask` — one application = one task

`CUserProcessTask` (subclass of `CTask`, **256 KB** stack — or what the app's folder's `app.txt` asks for, `stack = 8M`: `AppStackSize` reads it in the launcher's context before the task exists, since `CTask` allocates its stack in its constructor; rounded up to 64 KB, at most 64 MB; NetSurf asks for 8 MB):

1. Creates a fresh `CAddressSpace`.
2. Installs stdin/stdout, the process handle, `argv`, `cwd`.
3. `LoadELF` into the address space.
4. `SetUserData(AS, TASK_USER_DATA_USER) + AS->AddTask(this)` (its main task) + `Activate()` (switches `TTBR0/ASID`).
5. **Calls the entry point directly:** `((void(*)())entry)()` — in EL1, in the app's page table + stack. No trap.
6. On return, `ThreadsEndProcess()` (its other threads end with it) and `Terminate()`; the reaper reaps the tasks and frees the address space with the last one.

Threads (v67)

Source: `kernel/sys/thread.cpp`, `kernel/include/kern/thread.h`.

A thread is a `CUserThreadTask`: one more `CTask` whose `TASK_USER_DATA_USER` is the app's `CAddressSpace` (set in its constructor, before it is first scheduled: no `Yield` in between), so the task switch activates the app's page table and every `kapi` sees the same process — window, heap, files, sockets, `cwd`. It calls `fn` (`arg`) at EL1 on its own stack (kernel heap: the identity region, mapped in every space — like the main task's; 256 KB by default, 16 KB .. 16 MB), and the timer preempts it in its own code like any app (§5). `fn`'s return, or `kapi_thread_exit`, records its exit code and ends the task; the reaper frees it while the process goes on. A process runs at most `THREADS_MAX` (32) threads besides its main one.

- **The process ends with its main task** (return from `main`, `kapi_exit`), or when any thread calls `kapi_exit`: `ThreadsEndProcess` → `TerminateGroup` (all but the caller). A kill (task manager, `kill_pid`, an orphan) ends the group the same way (§5). A fault anywhere still halts the machine (the post-mortem console, §13), as before.
- **The per-process state** — `CProcThreads`, made on first use, freed by `~CAddressSpace` (`ThreadsFree`, first): the thread records (`tid` → task, done, exit code; 64: the ended ones are kept until joined, the oldest reused first), the synchronisation objects (256 handles), the posted-call ring (256) and two events: `DoneEv` (a thread ended: its joiners) and `WakeEv` (a post, a window event, the close box: `pump_wait`).
- **The objects** — mutex (recursive, an owner; released when its owner thread ends), event (manual or auto reset), barrier (a count, a round number). A kapi runs on core 0 and is not preempted, so an object's check-then-wait is atomic: the objects need no lock. A waiter blocks on the object's `CSynchronizationEvent`, only ever set-then-cleared (Circle's `Pulse` is private) — a condition variable's broadcast; it counts itself in `nWaiters` across the wait and checks its condition again when woken. An object closed while waited on (`sync_close`) is freed by its last waiter (they return -2); those left at the process's end are pulsed — unlinking the killed tasks still on their wait list, which `~CSynchronizationEvent` asserts is empty — and deleted.
- **Posts**: `post (fn, ctx, value)` appends to the ring and pulses `WakeEv`; `pump_events` runs the pending posts (at most one ring's worth) before the window's events, on the calling thread. `pump_wait` points the window's wake-up (`CWindow::SetWake: PushEvent` and `RequestExit` pulse it) at `WakeEv`, sleeps if nothing is pending, then pumps; `wait_for_exit` sleeps on it too (16 ms at most, as before).
- **Word waits (v68, a futex)** — `wait_word (addr, expected, ms) / wake_word (addr)`. The word's **physical** address is the key (`WordPhys`: AT S1E1R on the caller's `TTBR0`, then `PAR_EL1`; below 4 GB, the kernel's identity map): the same word of a shared surface, mapped at another address in each process, wakes across processes. A waiter (`TwordWaiter`: the address, the value, its process, a `CSynchronizationEvent`) lives on its task's stack, linked in one kernel list under an IRQ spin lock; it is linked, then the word read again, then it waits — with the IRQ masked until it is on the event's list (Circle's `Wait` tests then blocks: a tick's `Set` in between would be lost; `Yield` keeps each task's own `DAIF`). `wake_word` sets the events of the waiters on that address. **Code on an app core changes words without any kapi**: `WordWaitTick`, called by `PeriodicTick` in the 100 Hz timer interrupt, reads every sleeping word through the identity map and wakes those whose value moved (nothing to do, and nothing read, while nobody sleeps). Spurious wakes are allowed (callers loop). A dying process unlinks its waiters first (`ThreadsFree` → `WordWaitsFree`: a killed task's record is on its stack, freed after the batch's handlers). Test: `/bin/futextest`.
- **Priority (v68)**: `thread_priority (tid, 1)` makes a thread "real time" (§5): picked first whenever it is ready, as long as it sleeps before its slice ends — for an audio pump.
- **The lists show a process once**: `list_windows`, `list_tasks` and `list_procs` skip the tasks that are not its main task; a thread's name is `<app>:<name>` (or `<app>:<tid>`), which the crash reports' task lists show. `kill / kill_pid` refuse the caller's own process.
- **User side**: `umm_malloc / umm_free` take a lock (`kapi_lock`: an exclusive swap, a `yield` while it is held — a spin on an app core, which makes no kapi call), and `user/libc/onyx_syscalls.c` defines newlib's retargetable locks (the 8 static ones and the 10 `__retarget_lock_*` functions, which keeps newlib's `lock.o` out); a recursive lock's owner is the `tid` on core 0, the core on an app core. `errno` stays shared.

Launch entry points

- **LaunchApp(name)**: builds `SD:apps/<name>.app/main`, reads it into RAM, creates a `CUserProcessTask` (without stdio). Exposed via `kapi_launch`.
 - **SpawnProcess(path, args, stdin, stdout, cwd)**: creates a `CProcess` handle, increments the streams' refs, creates the task. Exposed via `kapi_spawn` (the terminal for pipes/redirections). `kapi_wait/kapi_proc_done` query the handle.
 - **ExecPath(path, args)**: "fire-and-forget" (without stdio or handle; the task name is derived from the path). Exposed via `kapi_exec` (the file manager to open a document in an app, or launch a program).
 - **Only ELF programs**: the kernel no longer knows BASIC (the former `BasicRedirect`). The formats a program runs (`.bas / .bax` → `SD:/bin/basic`) are chosen in user space: `SD:/etc/runners.ini` + `user/launch.h`, which starts the runner with `kapi_exec_as (path, args, name)` (v49, `ExecPath` with a name), so the process keeps the app's name (windows, `raise_app`, `list_windows`).
-

8. The kapi ABI table

Source: `kernel/include/kern/kapi_abi.h`, `kernel/sys/kapitable.cpp`, `kernel/sys/kapi.cpp`. App side: `user/kapi.h`.

The mechanism

Rather than linking the apps against the **kernel symbol addresses** (which move on each rebuild), the kernel publishes a **function-pointer table** (`struct TKApiTable`) at a **fixed virtual address**:

- `KAPI_TABLE_VA = 14 GB` — stable "forever" (between the canvas at 12 GB and the stack at 16 GB).
- The table is a static variable aligned to 64 KB (`s_Table` in `kapitable.cpp`), hence in the identity region (`PA == kernel VA`). `KApiTableInit()` fills all the pointers + the `version` field.
- Each `CAddressSpace` maps this page **read-only** at `KAPI_TABLE_VA` (cf. §4).
- App side, `kapi.h` defines `#define KT ((const struct TKApiTable *) KAPI_TABLE_VA)` and one inline function per entry (`kapi_create_window`, `kapi_open`, ...) that does nothing but dereference the table.

Consequence: an application binary **embeds no kernel address** and keeps working against any kernel that exposes the same ABI → no rebuild of the apps when the kernel changes.

The *append-only* contract

`KAPI_ABI_VERSION = 68`. The `TKApiTable` struct is **strictly append-only**: you never remove or reorder a field; you add new ones **at the end** and you increment the version. An old app only touches the prefix it knows → it stays compatible. The history of additions is annotated in the file (v1 = `app_dir`, v2 = `set_click_handler`, v3 = `opendir/readdir`, v4 = `streams/spawn`, ... v16 = `set_window_theme`, v17 = `chdir/getcwd`, v18 = `stdin_stream/stdout_stream`, v19 = `klog_read`, v20 = `set_verbose/get_verbose`, v21 = the TCP socket calls, v22 = `set_pointer_handler`, v23 = `meminfo`, v24 = `sbrk`, v25 = `reboot`, v26 = `kbd_ready`, v27 = `set_keymap_data`, v28 = `get_chrome/draw_text_buf` (user-side window chrome), v29 = compat break (kernel-drawn widget API removed, table consolidated), v30 = `random` (hardware RNG), v33 = `ram_detail`, v34 = `set_wheel_speed/get_wheel_speed`, v35 = shared surfaces (`surface_*`) + shell IPC (`register_shell`, `shell_request`, `mailbox_send/mailbox_recv`), v36 = `memset/memcpy/memmove`, v37 = `tcp_listen/tcp_accept`, v38 = `screen_grab/inject_pointer/inject_key`, v39 =

set_menu/get_menu/menu_command, v40 = ipc_register/ipc_lookup, clipboard_set/clipboard_get, set_window_alpha, shutdown, v41 = fullscreen_begin/present_fb/fullscreen_end, v42 = drag_begin/drag_data, get_modifiers/inject_modifiers, v43 = net_ping/net_resolve/net_info, v44 = vfs_register/vfs_next/vfs_req_data/vfs_reply, v45 = wlan_scan, v46 = sound_acquire/sound_release/sound_start/sound_stop/sound_write/sound_status, v47 = sound_instrument, v48 = key_held/inject_key_held, v49 = exec_as, v50 = pad_state, v51 = core_acquire/core_run/core_state/core_release, v52 = gpu_info/gpu_draw, v53 = gpu_texture/gpu_render, v54 = kapi_gpu_vertex3 second (added) colour r2 g2 b2 a2 in place of reserved, KAPI_GPU_B_ALPHATEST(t), v55 = fullscreen_direct, v56 = win_list/win_read/win_raise/win_close, v57 = seek, v58 = code_alloc, v59 = fsize64, v60 = sound_volume/wlan_reconnect, v61 = gpu_program/gpu_render2, v62 = gpu_render3, v63 = gpu_vbuf, v64 = win_minimise/win_geometry/resize_window2 — the modernised CDE desktop's windows: the frame's metrics and title buttons shared with the apps (KAPI_FRAME_*), GUI_EVENT_WINCTL, KAPI_WIN_MINIMISED, the frames' rounded corners, WIN_FLAG_ALPHA), v65 = desk/win_desk — the workspaces (virtual desktops: KAPI_WIN_OFFDESK, KAPI_WIN_DESK, KAPI_DESK_MAX), v66 = screen_set — the resolution changed while running, GUI_EVENT_DISPLAY_RESIZE; a window as big as the screen (was 1024 × 768 at most), v67 = thread_create/thread_exit/thread_join/thread_self, mutex_*/event_*/barrier_*/sync_close, post/pump_wait — threads (§7), their synchronisation objects, calls posted to the event pump; the scheduler's task list has no limit, v68 = sound_config/sound_map — low-latency sound and the mapped PCM ring (§12), wait_word/wake_word — a futex (§7), thread_priority — "real time" threads (§5), midi_read/midi_devices — USB MIDI input (struct kapi_midi_event).

The callbacks' value (gui_handler: sender, event, value) is the type gui_value: long on Onyx (64 bits: a pointer event packs its wheel, buttons and position there), long long where long has 32 bits -- the Windows build of the apps (pc/Koton, docs/03 Koton for Windows), whose table is filled by a Win32 layer. On Onyx it is the same type as before: no ABI change, no new version.

Categories of exposed functions

Category	Examples
Windowing	create_window(_ex) (the canvas; 0 when the client area is bigger than the screen — g_nScreenWidth/Height; before v66, 1024 × 768 — or memory is short — an app must check it: at EL1, a null canvas drawn into is the kernel's memory at address 0), resize_window (the client size shown, ≤ the canvas made at creation; the frame — OuterW/H, the chrome copies' size — follows it, and the app redraws its chrome: wk_decorate_window), move_window, present, exit. Window flags: WIN_FLAG_BORDERLESS, WIN_FLAG_BACKMOST (desktop, bottom band), WIN_FLAG_TOPMOST (the menu bar: top band, never the active app nor the key target; at y=0 it reserves its smallest logical height —

Category	Examples
	<code>CWindowManager::TopInset()</code> — so auto-placement and title-bar drags stay below it), <code>WIN_FLAG_TRANSPARENT</code> (client blitted with the magenta key), <code>WIN_FLAG_SYSTEM</code> (a shell component — menu bar, notifications, panel, app list: skipped by <code>list_windows</code>, so never in the taskbar; a plain flag bit, no ABI change). The z-order is three bands: <code>backmost < normal < topmost</code> (<code>Add/RaiseLocked</code> keep them). The key target is the frontmost non-topmost window; the active app (menus, chrome highlight uses the key target) is the frontmost window that is neither topmost, backmost nor borderless.
Menu bar (v39)	<code>set_menu(spec, handler)</code> stores the app's menu spec (≤ 2 KB; lines <code>M<title>, I<id>\t<label>\t<shortcut>, -)</code> + a <code>GUI_EVENT_MENU</code> (14) handler on its <code>CWindow</code>; <code>get_menu(buf, cap, title, tcap)</code> returns the active app's spec + title and a serial that changes with the active window or its menu (0 = none); <code>menu_command(id)</code> queues <code>GUI_EVENT_MENU(id)</code> to the active window (<code>MENU_QUIT = -1</code> → <code>RequestExit</code>, like the close box). Used by <code>menubar</code> + <code>wtk::Menu</code>.
Launch/management	<code>launch</code>, <code>toggle_app</code>, <code>raise_app</code>, <code>exec</code>, <code>kill</code>, <code>kill_pid</code>
Threads (v67)	<code>thread_create(fn, arg, stack_size, name)</code> → <code>tid</code> ≥ 2 (main: 1), -1 no memory, -2 too many (32); <code>thread_exit(code)</code> (the main thread: the process); <code>thread_join(tid, timeout_ms, &code)</code> → 0, -1 timeout, -2 none / joined already, -3 itself; <code>thread_self</code>. <code>mutex_create/mutex_lock(h, timeout)/mutex_unlock (recursive)</code>, <code>event_create(manual, initial)/event_set/event_reset/event_wait(h, timeout)</code>, <code>barrier_create(count)/barrier_wait</code> (1 for the last one in), <code>sync_close</code> —

Category	Examples
	handles, 256 per process; timeouts in ms, 0 = only try, <code>KAPI_WAIT_FOREVER</code> . <code>post(fn, ctx, value)</code> → queued for the pump (−1 full: 256); <code>pump_wait(timeout)</code> sleeps until an event / a post / the close box, pumps → what was pending. See §7.
Word waits, priority (v68)	<code>wait_word(addr, expected, timeout_ms)</code> sleeps while the 4-byte word <code>*addr == expected</code> → 0 (woken, or the value differs), 1 timeout (0 ms: only check), −1 bad address (unaligned, unmapped); <code>wake_word(addr)</code> → the sleepers woken. Keyed by the physical address (a word of a shared surface wakes across processes); the 100 Hz tick also reads every sleeping word and wakes those that changed — an app core's write needs no <code>wake_word</code> (≤ 10 ms). <code>thread_priority(tid 0 self / 1 main / ≥ 2, prio 0 / 1 / −1 ask)</code> → the previous one, −1 bad prio, −2 no such thread: a "real time" task is picked first when ready and a tick preempts an app for it, while it yields by itself (§5). See §7.
Enumeration	<code>list_apps</code> , <code>list_windows</code> , <code>list_tasks</code> , <code>list_procs</code> , <code>get_datetime</code>
Widgets	<code>add_button/label/checkbox/textbox/progress/slider/textarea/scrollbar/icon</code> , <code>widget_get/set_*</code>
Events	<code>pump_events</code> , <code>wait_for_exit</code> , <code>should_exit</code> , <code>set_key_handler</code> , <code>set_click_handler</code> , <code>set_pointer_handler</code> (full pointer stream, v22 — incl. <code>GUI_EVENT_PTR_WHEEL</code> , a signed scroll-notch delta in the <code>lValue</code> wheel field via <code>GUI_PTR_WHEEL</code>)
App-drawn text	<code>draw_text</code> , <code>font_width</code> , <code>font_height</code>
Files	<code>open/read/fsize/close</code> , <code>save_file</code> , <code>opendir/readdir/closedir</code> , <code>mkdir/remove/rename</code> , <code>chdir/getcwd</code> (current working directory, inherited by children). <code>fsize</code> (and <code>readdir</code> 's size) is

Category	Examples
	clamped to 4 GB – 1; <code>fsize64(h)</code> (v59) gives an exFAT file's real 64-bit size. <code>rename</code> across two volumes fails (–1): the caller copies then deletes (FatFs' <code>f_rename</code> would otherwise rename inside the source volume).
Volumes (v59)	FatFs volume strings (<code>FF_STR_VOLUME_ID</code> , docs/05 §13): SD: = the SD card's first FAT volume (partition 1, the boot FAT32 one, found as before; SD0: is an alias), SD1: SD2: SD3: = MBR partitions 2–4 (<code>FF_MULTI_PARTITION</code>) (FAT12/16/32 or exFAT , mounted at boot when present), USB:... , FD: , NVME: (declared, not mounted yet). <code>ResolvePath</code> : a volume prefix is upper-cased (<code>sd1:</code> → SD1: , SD0: → SD:), a path starting with <code>/</code> is relative to the current directory's volume root, anything else to the current directory. The four SD volumes share one FatFs lock slot (<code>LockSlot</code> , <code>sys/fslock.cpp</code>): they are one card, one command at a time.
Streams/processes	<code>pipe</code> , <code>file_in/out</code> , <code>stream_read(_nb)/write/close/eof</code> , <code>stdin_read</code> , <code>stdout_write</code> , <code>spawn</code> , <code>wait</code> , <code>proc_done</code> , <code>get_args</code>
Modal dialogs	<code>message_box</code> , <code>file_open</code> , <code>file_save</code>
Desktop	<code>screen_size</code> , <code>set_wallpaper</code> , <code>wallpaper_generate</code> , <code>wallpaper_buffer</code> , <code>wallpaper_commit</code> , <code>cursor_pos</code>
Appearance/keyboard	<code>set_window_theme</code> , <code>set_keymap</code> (load a country map <i>by name</i> — deprecated: the kernel compiles in no maps, so it always returns 0; use <code>set_keymap_data</code>), <code>get_keymap</code> , <code>kbd_ready</code> (USB keyboard attached? v26 — informational; <code>keyb</code> no longer needs to poll it), <code>set_keymap_data</code> (load a layout from a <code>.kmap</code> blob, v27 — records it in a persistent snapshot and installs it on the keyboard whenever it attaches, so it needs no keyboard to be present; returns 1 once the blob is accepted, 0 only on a malformed blob. This

Category	Examples
	removed the old boot race where <code>keyb</code> could time out waiting for USB enumeration and leave the keyboard map-less), <code>app_dir</code>
Logging / memory	<code>klog_read</code> , <code>set_verbose</code> , <code>get_verbose</code> , <code>meminfo</code> (total/free/app KB + page size, v23), <code>sbrk</code> (per-process heap, v24)
Networking (v21, v37)	<code>net_status</code> (live: 0 as soon as the Wi-Fi association drops — <code>CWPASupplicant::IsConnected ()</code> — not only before the first DHCP bind), <code>tcp_connect</code> , <code>tcp_send</code> , <code>tcp_recv</code> , <code>tcp_close</code> ; server side (v37): <code>tcp_listen(port)</code> → listening handle (<code>Circle CSocket::Bind+Listen</code> ; -6 = port in use), <code>tcp_accept(h, ip, cap)</code> → blocks until a peer connects, returns a connected handle + the peer IP (<code>Circle Accept</code>). All handles share the 16-slot table in <code>sys/net.cpp</code> and are reclaimed when the owner dies. Used by <code>/bin/telnetd</code> .
Power (v25)	<code>reboot</code> (restart the machine — applies settings read only at boot, e.g. the WLAN config rewritten by <code>wpaconf</code>)
Remote screen (v38)	<code>screen_grab(dst, w, h)</code> — runs <code>CWindowManager::Composite</code> (windows, wallpaper, cursor; <code>bCountFrame = FALSE</code> so the watchdog's fps stays the real compositor's) straight into the caller's <code>w*h 0x00RRGGBB</code> buffer (<code>w/h</code> must be the screen size), or returns 2 without touching it when <code>g_nScreenGen</code> has not changed since the previous grab into the same buffer (<code>vncd</code> then skips the diff / encode); <code>inject_pointer(x, y, buttons, wheel)</code> → <code>OnMouse</code> (+ <code>OnMouseWheel</code>), <code>inject_key(keys)</code> → <code>OnKey</code> — the same paths as the USB mouse/keyboard. Used by <code>/bin/vncd</code> .
Windows as objects (v56)	For the window-level remote desktop (<code>/bin/rdpd</code>): <code>win_list(out, max)</code> fills <code>struct kapi_win_info</code> bottom to top (<code>CWindowManager::Snapshot</code>): <code>id</code> (<code>CWindow::Id</code> , a serial never reused),

Category	Examples
	owner pid, the client area on the screen (x y w h; the full-screen window: the whole screen, <code>KAPI_WIN_FULLSCREEN</code>), <code>WIN_FLAG_*</code>, alpha, <code>gen</code> (<code>CWindow::Gen</code>, bumped by every <code>Damage</code> — drawn, moved, resized, raised — and by <code>present_fb</code>), <code>KAPI_WIN_KEYS</code> (the key target), the title, the frame (<code>ow oh = OuterW/H, il it = the insets, 0</code> without a frame) and <code>chromeGen</code> (bumped by <code>get_chrome</code>, i.e. when the app redraws its frame, by a resize, and once more by the first present after that: the frame read again whole). The state also says <code>KAPI_WIN_MINIMISED / KAPI_WIN_OFFDESK</code> (not shown: <code>rdpd</code> sends a hidden window's pixels only when it is shown again) and the window's desk (<code>v65</code>). <code>win_read(id, part, x, y, w, h, dst, stride)</code>: a rectangle of the canvas (part 0; the full-screen window: its back buffer, or the screen itself when direct) or of a chrome copy (1 active, 2 inactive), clipped. <code>win_raise(id) = Raise</code>, <code>win_close(id) = RequestExit</code> (as the close box). The desktop is listed first as a pseudo-window, <code>KAPI_WIN_DESKTOP</code> (<code>0xFFFFFFFF</code>, backmost, screen-sized, <code>gen</code> = the wallpaper's counter + the backmost windows'): <code>win_read</code> of it (whole only) runs <code>CWindowManager::CompositeDesktop</code> (the wallpaper + the backmost windows) straight into the caller's buffer. A <code>CWindow</code> is never freed (see <code>Composite</code>), so an id found in the snapshot is safe to read.
Generated code (v58)	<code>code_alloc(size)</code> — zeroed memory the app may write and execute (a JIT's code: <code>gcemu</code>'s PowerPC → AArch64 translation), rounded up to 64 KB pages: <code>CAddressSpace::CodeAlloc</code> maps fresh owned frames (<code>MapNewPage</code>, freed at teardown) with <code>KPAGE_ATTR_APP_RWX</code> (EL1 RW, PXN = 0) in a per-process

Category	Examples
	bump arena <code>USER_CODE_BASE</code> (14.25 GB) ... <code>USER_CODE_END</code> (15 GB, 768 MB); 0 when full. Circle clears <code>SCTLR_EL1.WXN</code>, so a writable page may be executable. After writing code the app cleans the D-cache and invalidates the I-cache over it (<code>DC CVAU</code>, <code>DSB ISH</code>, <code>IC IVAU</code>, <code>DSB ISH</code>, <code>ISB</code>).
File seek (v57)	<code>seek(h, pos)</code> — the read position of a file opened with <code>open (f_lseek)</code>; 0, or <code>-1</code> (a VFS provider's file: not seekable). A file bigger than 4 MB gets its cluster map at its first seek (FatFs fast seek, <code>CREATE_LINKMAP</code>, docs/05 §11), freed by <code>close</code>: any position without walking the FAT. For <code>gcemu</code>'s disc images (1.4 GB, read on demand).
Shared surfaces (v35)	<code>surface_create(w, h)</code> → an <code>id > 0</code>: a page-aligned, physically contiguous <code>0x00RRGGBB</code> buffer (<code>CSurface</code>, <code>kern/gui/surface.h</code>) owned by the caller; <code>surface_map(id)</code> maps the same frames into the caller (the <code>id</code> travels to another process over IPC) → their address; <code>surface_size</code>, <code>surface_present</code> (yields toward the process composing it), <code>surface_destroy</code>. A process mapping a surface it does not own becomes one of its users (v65, ≤ 4): the frames are freed once the owner let it go (<code>surface_destroy</code>, or its end: <code>DestroyByOwner</code>) and every user has ended too — no process keeps a mapping of freed frames, whichever ends first. The Control Panel's applets draw into their host's surface this way (<code>user/applet_proto.h</code>, the developer guide).
IPC services (v40)	<code>ipc_register(name)</code> makes the caller the service <code>name</code> (≤ 16 services; a name held by a live process is refused; freed when the owner dies — <code>IpconProcessGone</code>); <code>ipc_lookup(name)</code> → <code>pid</code> or 0. Messages use the per-process mailboxes (<code>mailbox_send/mailbox_recv</code>), now up

Category	Examples
	to 512 bytes (<code>MAILBOX_MSG_MAX</code>). The kernel itself can post: <code>IpNotify(title, text)</code> → the <code>notify</code> service (e.g. "Network ... connected").
Clipboard (v40)	<code>clipboard_set(type, data, len)</code> / <code>clipboard_get(&type, buf, cap, &serial)</code> : one typed blob (≤ 64 KB) held by the kernel so it outlives the app that copied (<code>CLIP_TEXT</code> 1, <code>CLIP_FILES</code> 2, <code>CLIP_FILES_CUT</code> 3 — paths <code>\n</code> -separated); the serial bumps on every set.
Window opacity / session (v40)	<code>set_window_alpha(0..255)</code> on the caller's window: <code>CWindow::DrawTo</code> blends chrome + client over what is below (<code>BlendRect</code> , magenta-keyed if <code>TRANSPARENT</code> ; 0 = not drawn) — used for fades. <code>shutdown(mode): f_mount(0)</code> unmounts/flushes the SD card, then <code>reboot()</code> (mode 1) or ACT LED off + <code>halt()</code> (mode 0).
Full-screen apps (v41)	<code>fullscreen_begin(&w, &h)</code> maps a kernel-owned, screen-sized back buffer (<code>CWindowManager::EnsureFullscreenBuffer</code> , 64 KB-aligned) at <code>USER_FULLSCREEN_CANVAS</code> (15 GB) and makes the caller's window (created if missing, moved to 0,0) the full-screen window : the compositor task skips its frames, <code>OnMouse/OnMouseWheel</code> send the whole pointer stream to it in screen coordinates and it is the key target. <code>present_fb()</code> copies the buffer into the displayed <code>C2DGraphics</code> buffer + <code>UpdateDisplay()</code> , then yields. <code>fullscreen_end()</code> — or the window's removal when the app exits — gives the desktop back. <code>screen_grab</code> (VNC) returns the full-screen buffer meanwhile. v55 <code>fullscreen_direct(&w, &h, &stride)</code> (after <code>fullscreen_begin</code>): maps the framebuffer the display scans out (<code>CBcmFrameBuffer</code> , the <code>C2DGraphics</code> display, 32 bpp) at <code>USER_FULLSCREEN_SCREEN</code> (15.5 GB), normal uncached

Category	Examples
	(KPAGE_ATTR_APP_SCREEN); <code>present_fb</code> then copies nothing (it only yields); <code>screen_grab</code> (VNC) reads the screen through the same uncached mapping in the grabber's address space (the kernel's own map of the framebuffer is Device memory, slow to read). What is drawn there shows at once (no double buffering: tearing is possible); the GPU renders there directly (below 1 GB, contiguous), so a full-screen GPU frame costs no copy at all. Returns 0 when not possible (keep the back buffer).
Drag & drop (v42)	<code>drag_begin(type, data, len, label)</code> — only while the left button is held — copies the payload (≤ 4 KB: 1 text, 2 \n-separated paths) into a kapi-side buffer and calls <code>CWindowManager::DragBegin(src, label)</code>. While the session lasts, <code>OnMouse</code> sends <code>GUI_EVENT_DRAG_OVER</code> (16) to the window under the cursor (<code>DND_F_LEAVE</code> when it leaves), and <code>Composite</code> draws a label badge next to the cursor (a + when Ctrl is held). At the left-button release, <code>DndFinishLocked</code> sends <code>GUI_EVENT_DROP</code> (15) to the window under the cursor — <code>(flags << 32) (x << 16) y</code>, client coords, <code>DND_F_COPY</code> if Ctrl — and <code>GUI_EVENT_DRAG_DONE</code> (17) to the source: <code>(flags << 32) target pid</code> (<code>CWindow::OwnerPid</code>, set by <code>CreateWindow</code>), flags <code>DND_F_COPY</code> / <code>DND_F_CANCEL</code> (Esc, via <code>OnKey</code>) / <code>DND_F_DESKTOP</code> (no window or a backmost one). All three go to the pointer handler; the normal pointer stream still reaches the source (its capture), so its widgets see the button go up. The target reads the payload with <code>drag_data(&type, buf, cap)</code>. A window removed mid-drag ends it. Modifiers: <code>get_modifiers() = MOD_CTRL 1 / MOD_SHIFT 2 / MOD_ALT 4</code> — from the USB keyboard's raw report (<code>RegisterKeyStatusHandlerRaw</code> in mixed mode, so the cooked key path is

Category	Examples
	unchanged) or <code>inject_modifiers()</code> (vncd, from the RFB Control/Shift/Alt keysyms). Per key event: <code>OnKey</code> stores the modifiers in each <code>GUIEvent</code> (<code>nMods</code> = the global state OR the xterm parameter of <code>ESC[1;<m>X / ESC[n;<m>~</code>, <code>m - 1</code> = Shift 1 + Alt 2 + Ctrl 4, which Circle's keymap and vncd send for navigation keys); <code>kapi_pump_events</code> sets <code>CWindow::m_nKeyEventMods</code> around the app's key handler, and <code>get_modifiers()</code> returns it while the handler runs — so Shift+arrow selects even when the live state is late or clobbered (VNC, a second keyboard).
Network tools (v43)	<code>net_ping(host, seq, timeout_ms, ip, cap)</code> resolves the host (<code>CDNSClient</code> unless a dotted quad), builds an ICMP echo request (id <code>0x4F4E</code>, 32 data bytes, <code>CChecksumCalculator</code>) and sends it with <code>CNetworkLayer::Send(..., IPPROTO_ICMP)</code>; the reply is read from Circle's secondary ICMP queue (<code>EnableReceiveICMP</code>, enabled only while a ping is in flight, <code>ReceiveICMP</code>), matching type 0 + id + seq + sender, yielding while it waits → RTT in μs or -1 down / -3 unresolved / -4 timeout / -5 send failed. <code>net_resolve</code> = DNS only. <code>net_info</code> = a text dump for <code>netstat</code>: up, hostname, ip, mask, gateway, dns, dhcp lines (<code>CNetConfig</code>), then one tcp <handle> listen conn <local port> <remote ip> <pid> per socket slot (<code>TSocketSlot.bListen</code>). Tools: <code>/bin/ping</code>, <code>nslookup</code>, <code>netstat</code>, <code>whois</code> (the latter is plain TCP port 43).
User-space file systems (v44)	<code>sys/vfs.cpp</code>, <code>kern/vfs.h</code> — FUSE-like. A provider app calls <code>vfs_register(prefix)</code> (<code>/bin/ftpfs</code>: FTP:, FTPS:). <code>kapi_open/read/fsize/close</code>, <code>save_file</code>, <code>opendir/readdir/closedir</code>, <code>mkdir/remove/rename</code> on a path with a registered prefix (<code>VfsHandles</code>) become requests (<code>VfsCall</code>): the calling task fills

Category	Examples
	a slot (op, path, path2, a0–a2, a kernel copy of the payload), sets the provider's <code>CSynchronizationEvent</code> and waits on the slot's own event (200 ms re-checks: provider alive via <code>IpcPidAlive</code>, 120 s timeout). The provider takes it with <code>vfs_next</code> (blocking = up to 0.5 s, so it can also poll its mailbox), reads the payload with <code>vfs_req_data</code>, answers with <code>vfs_reply(id, status, data, len)</code> (copied into a kernel buffer, then into the caller's). Ops: <code>OPEN</code> (→ fid + size), <code>READ</code> (fid, offset, ≤ 64 KB), <code>CLOSE</code>, <code>LIST</code> (packed u32 size, u8 is_dir, name\0 entries), <code>SAVE</code>, <code>MKDIR</code>, <code>REMOVE</code>, <code>RENAME</code>. Provider-backed handles live in static tables (<code>s_File</code>, <code>s_Dir</code>), so the file kapis tell them from FatFs <code>FIL/DIR</code> by address. <code>FTP:/FTPS:</code> are auto-started: the first use execs <code>SD:/bin/ftpfs</code> and waits up to 5 s for it to register. A dying provider is dropped and its pending requests fail (<code>VfsOnProcessGone</code>, from <code>IpcOnProcessGone</code>). Streams (<code>kapi_file_in</code>: cat, redirections) still go to FatFs only.
Wi-Fi scan (v45)	<code>wlan_scan(out, max)</code> → struct <code>kapi_wlan_ap</code> (ssid, bssid, security <code>WLAN_SEC_OPEN/WEP/WPA/WPA2</code>, channel, freq, level dBm, connected), strongest first, one per BSSID. <code>NetWlanScan</code> in <code>sys/net.cpp</code> — no Circle patch: it drives the BCM4343 firmware's <i>escan</i> through <code>CBcm4343Device::Control("escan 5")</code>, collects <code>ReceiveScanResult</code> messages for ~3.5 s (the firmware's <code>brcmf_escan_result_le</code> layout, as in hostap's <code>driver_circle.cpp</code>), then <code>escan 0</code>. Security from the capability privacy bit + the RSN (48) / WPA vendor (221) IEs; <code>connected</code> = the BSSID <code>GetBSSID()</code> reports while <code>CWPASupplicant::IsConnected()</code>. <code>wpa_supplicant</code> reads the same result queue for its own scans: while it is still

Category	Examples
	looking for its network, a scan here may take its results (it scans again). Used by <code>/bin/wifiscan</code> and <code>wpaconf</code> .
Master volume (v60)	<code>sound_volume(volume 0..10, mute 0/1)</code> (–1 keeps a value) → the volume`
Wi-Fi join (v60)	<code>wlan_reconnect()</code> — <code>NetWlanReconnect</code> (a core-3 request with <code>netcore=1</code>): <code>wpa_supplicant</code> 's <code>SIGHUP</code> handler, caught at link time (– <code>wrap=eloop_register_signal_reconfig, kernel/Makefile; docs/05 §14</code>) and run from its own event loop (a 0 s <code>eloop</code> timeout) — deauthenticate, read <code>SD:/etc/wpa_supplicant.conf</code> again, rescan, join the highest priority network in range — then <code>CDHCPClient::Restart()</code> (a new lease: another network). 0 asked, –1 no Wi-Fi running. Used by the Wi-Fi menu (<code>wifimenu</code>).
Low-latency sound (v68)	For the sound owner: <code>sound_config(chunk_frames 64..1024 (0: 1024), ahead 1..4 (0: 4))</code> → the latency now in frames, <code>(ahead + 1) × chunk</code> (–1 not the owner); <code>sound_map()</code> → the mapped PCM ring (<code>struct kapi_sound_ring</code> : one 64 KB page, 8192 frames, <code>wr</code> moved by the app after its frames, <code>rd</code> by the kernel, <code>dry</code> underruns, the chunk / ahead now), mixed with the voices and the stream until the owner releases the output — plain memory, so an app core fills it (<code>kapi_sound_ring_write</code>). Both are back to the defaults / off at <code>sound_release</code> or the owner's death. See §12. Test: <code>/bin/ringtest</code> .
Sound (v46)	<code>sound_acquire</code> (1 ok / 0 busy / –1 no audio: the caller becomes the owner; the output starts on first use), <code>sound_release</code> , <code>sound_start(voice 0..15, milliHz, wave SOUND_SQUARE/SINE/TRIANGLE/SAW/NOISE, volume 0..255)</code> (plays until stopped), <code>sound_stop(voice or -1)</code> , <code>sound_write(s16 stereo frames, n)</code> → frames taken (PCM ring, non-blocking),

Category	Examples
	<code>sound_status(&rate, &free, &owner)</code> . Non-owners get -1; the owner's exit silences it. See §12.
Run as (v49)	<code>exec_as(path, args, name)</code> → <code>ExecPath</code> with the process named <code>name</code> instead of after the path (1 = started). User space runs a format's program this way (<code>launch.h</code> : <code>SD:/bin/basic</code> for an app's <code>main.bax</code> is named after the app).
USB MIDI (v68)	<code>midi_read(ev, max)</code> → up to <code>max</code> <code>struct kapi_midi_event</code> (<code>time_us</code> — the kernel's μ s clock, <code>CTimer::GetClockTicks</code> , which <code>kapi_clock_us</code> reads user side, on an app core too —, <code>cable</code> , <code>status</code> , <code>data1</code> , <code>data2</code> , <code>device</code> (its <code>umidiN</code>), <code>length</code> 1..3), oldest first, never waits; -1 bad arguments. One queue for the system (256 events, the newest dropped when full). <code>midi_devices()</code> → attached. Circle's USB device factory makes a <code>CUSBMIDIHostDevice</code> for every class-compliant MIDI interface (<code>int1-3-0</code>) and its <code>CUSBMIDIDevice</code> names itself <code>umidiN</code> ; the input task finds <code>umidi1..4</code> every 100 ms (a hot plug too), registers its packet handler (<code>MidiPacket</code> : at USB-completion time, stamps the packet and queues it under an IRQ spin lock) and a removed handler that frees the slot. No Circle change. Test: <code>/bin/miditest</code> .
Gamepads (v50)	<code>pad_state(index, out)</code> → 1 and <code>struct kapi_pad</code> filled for USB gamepad 0..3 (<code>KAPI_PAD_MAX</code>), else 0: <code>vid/pid</code> , <code>props</code> (Circle's <code>TGamePadProperty</code> , bit 0 = a known mapping), <code>focus</code> (the caller's window has the keyboard), <code>seq</code> (reports received), <code>nbuttons/buttons</code> , <code>naxes/axes[16]</code> (value, min, max), <code>nhats/hats[6]</code> (0..7 = N..NW). Raw state: for pads Circle knows (Xbox 360 / One, PS3 / PS4, Switch Pro) <code>buttons</code> are its <code>TGamePadButton</code> bits, for other HID pads the report's own. The input task finds <code>upad1..4</code> (Circle's names) every 100 ms, registers a status handler that copies

Category	Examples
	each report into a slot under a sequence count (odd while writing: the handler runs at USB-completion time), and a removed handler that frees the slot. The mapping to one button set is user space (user/gamepad.h, SD:/etc/gamepad.ini).
App cores (v51)	<code>core_acquire()</code> → 2 or 3 (a free app core, now the caller's) or -1; <code>core_run(core, fn, arg, stack_top)</code> → 0, or -1 (not yours / still running / <code>fn</code> or the stack not a user address): the core calls <code>fn (arg)</code> in the caller's address space on the given stack (16-byte aligned, the caller's memory); <code>core_state(core)</code> → <code>KAPI_CORE_IDLE</code> (0: <code>fn</code> returned), <code>KAPI_CORE_RUNNING</code> (1), <code>KAPI_CORE_FAULT</code> (-2: <code>fn</code> faulted and was stopped, logged to <code>kmsg</code>) or <code>KAPI_CORE_NOTYOURS</code> (-1); <code>core_release(core)</code> stops <code>fn</code> if it runs and frees the core (done at the app's exit anyway). <code>fn</code> makes no kapi call and no allocation. See §14.
GPU (v52)	<code>gpu_info(buf, cap)</code> → 1 (the V3D is up; <code>buf</code> = "V3D 4.2 (1 core)") or 0 (<code>buf</code> says why); the first call brings the GPU up. <code>gpu_draw(v, n, clear, pixels, w, h, stride)</code>: <code>n</code> vertices struct <code>kapi_gpu_vertex { float x, y, z; u8 r, g, b, a; }</code> (a triangle list; normalized device coordinates, y up, z -1 near ... 1 far; depth test <i>less</i>, both faces; colours interpolated) rendered by the GPU into <code>pixels</code> (0x00RRGGBB, $w \times h \leq 2048$, <code>stride</code> pixels a row) after clearing it to <code>clear</code> (0xRRGGBB) → 0, -1 no GPU, -2 bad arguments / too many vertices (<code>KAPI_GPU_MAX_VERTS</code> = 196608), -3 the GPU did not finish (it is then left off). See §15.
GPU (v53)	<code>gpu_texture(handle, pixels, w, h, stride)</code>: a texture of $w \times h$ (≤ 2048) pixels 0xAARRGGBB; <code>handle</code> < 0 makes one (≤ 256 in all), ≥ 0 replaces its pixels, <code>pixels</code> = 0 frees it → the handle, -1 no GPU, -2 bad arguments, -4 no memory

Category	Examples
	/ no free handle; a program's textures are freed when it ends. <code>gpu_render(f, v, nv, b, nb)</code>: one frame into struct <code>kapi_gpu_frame { pixels, w, h, stride, clear, flags }</code> (<code>KAPI_GPU_F_KEEP</code>: drawn over the pixels instead of clearing them) of <code>nv</code> vertices struct <code>kapi_gpu_vertex3 { float x, y, z, w, s, t; u8 r, g, b, a; u8 r2, g2, b2, a2; }</code> (v54: the second colour is added after the texel $\times$ colour product and clamped to 1 — 0 for the v53 behaviour) in <code>nb</code> (≤ 4096) batches struct <code>kapi_gpu_batch { first, count, texture, flags, float matrix[16]; }</code>: each batch draws its triangles with its own matrix (row by row, clip = $M \cdot (x \ y \ z \ w)$, transformed by the GPU, which divides by <code>w</code> and clips; <code>KAPI_GPU_B_NOMATRIX</code> = identity), texture (<code>-1</code>: colour only, else texel $\times$ colour; <code>B_LINEAR</code>, <code>B_WRAP_S/T(REPEAT, CLAMP, MIRROR)</code>), depth test (<code>B_ZFUNC</code>: less by default, ... always) and writes (<code>B_NOZWRITE</code>), culling (<code>B_CULL_BACK/FRONT</code>, front = counter-clockwise, y up), alpha test (v54: <code>B_ALPHATEST(t)</code>, fragments whose $\alpha < t / 255$ are discarded) and blending (<code>B_BLEND(ALPHA, ADD, MUL, PREMUL)</code>) $\rightarrow 0, -1, -2, -3$ as <code>gpu_draw</code>. The kernel gives the V3D only well-formed triangles: each one is transformed by its batch's matrix on the CPU and clipped against the near plane ($z \geq -w, w > 0$) and a guard band $4 \times$ the screen (<code>kern/v3d_clip.h</code>, the batches then <code>NOMATRIX</code>; test <code>tools/tests/run_v3d_clip_test.sh</code>) — Ocarina of Time's triangles crossing the eye plane, projected $\sim 50\,000$ screens away, wedged the GPU and froze the whole Pi. See §15.
GPU (v61)	<code>gpu_program(handle, p)</code>: the app's own shaders — struct <code>kapi_gpu_program { vs, cs, fs; nvs, ncs, nfs; inputs, csInputs, csOutputs, varyings, flags }</code>, three V3D 4.2 QPU programs

Category	Examples
	(≤ 4096 instructions each; built at run time by <code>user/v3d/qpu.h</code>) copied into GPU memory; a vertex is <code>inputs</code> floats (4..64, the clip-space <code>x y z w</code> first; the coordinate shader reads the first <code>csInputs</code>, writes <code>csOutputs</code> ≥ 6: <code>Xc Yc Zc Wc Xs Ys</code>), the vertex shader writes <code>Xs Ys Zs 1/Wc</code> then <code>varyings</code> (≤ 64); flags <code>KAPI_GPU_P_FS_4WAY</code> (4 threads, else 2), <code>P_FS_FINAL</code> (no thread switch — unreliable on wide targets, see §15), <code>P_FS_ZWRITE</code>; <code>handle < 0</code> makes one (≤ 256), ≥ 0 replaces it, <code>p = 0</code> frees it $\rightarrow$ the handle, $-1, -2, -4$ as <code>gpu_texture</code>; freed when the program ends. <code>gpu_render2(f, v, nv, stride, b, nb, uni, nuni)</code>: one frame (as <code>gpu_render</code>) of <code>nv</code> vertices of <code>stride</code> floats in <code>nb</code> batches <pre>struct kapi_gpu_batch2 { first, count, program, flags, blend, wmask, scissor[4], vsUni, vsNUni, csUni, csNUni, fsUni, fsNUni, tex[8], texFlags[8], texUni[8] }; each with its program, its uniforms (three ranges of <code>uni[nuni]</code>, $\leq 2^{20}$ words), up to 8 textures — the kernel writes each one's TMU words <code>p0</code> (texture state, 16-bit float <code>RG / BA</code>) and <code>p1</code> (a sampler from <code>texFlags</code>: <code>B_LINEAR, B_WRAP_S/T</code>) at <code>fsUni + texUni[i]</code> —, depth / cull flags (<code>B_ZFUNC, B_NOZWRITE, B_CULL_*</code>), blending <code>KAPI_GPU_BLEND2(cSrc, cDst, aSrc, aDst, cEq, aEq)</code> (the V3D factors and equations), a colour write mask (<code>wmask</code>: the channels not written) and a scissor (<code>x y w h</code> in the target, top- left origin; <code>w ≤ 0</code>: none) $\rightarrow 0, -1, -2, -3$. The triangles are clipped on the CPU as for <code>gpu_render</code> (<code>V3DClipTriangleN</code>: every float of the vertex interpolated). See §15.</pre>
GPU (v62)	<pre>gpu_render3(f, v, nfloats, b, nb, uni, nuni, view): gpu_render2 with each batch's vertices where the app keeps them — struct kapi_gpu_batch3 { struct kapi_gpu_batch2 b; unsigned off, stride; }; b.count</pre>

Category	Examples
	vertices of <code>stride</code> floats ($\geq$ the program's inputs, ≤ 64) from float <code>off</code> of <code>v[nfloats]</code> (<code>b.first</code> unused) — and their <code>x / y</code> framed by the kernel on the way, <code>x' = view[0] x + view[1] w</code>, <code>y' = view[2] y + view[3] w</code> (<code>view 0</code>: as they are): no common array for the app to make first → as <code>gpu_render2</code>. <code>gcemu</code>'s TEV renderer gives it its recorder's frame as it is. See §15.
GPU (v63)	<code>gpu_vbuf(bytes)</code>: memory the GPU reads too — low, physically contiguous, mapped into the program (the surface arena); up to 8 a program, 64 MB each, freed when it ends → its address, 0 none. <code>gpu_render3</code> with <code>v[nfloats]</code> inside one draws the vertices where they are: their <code>x / y</code> framed in place (the program draws the same vertices again with <code>view 0</code>), the triangles inside every plane drawn as runs of them, the others clipped into the buffer's end past <code>nfloats</code> (keep room there) — nothing copied.
Windows (v64)	The modernised CDE desktop's windows. <code>win_minimise(id)</code>: window <code>id</code> (0: the caller's) minimised — not drawn, not hit, never active nor the keys' target — until <code>win_raise / raise_app</code> brings it back (<code>KAPI_WIN_MINIMISED</code> in <code>win_list</code>'s state) → 0 / -1. <code>win_geometry(out)</code>: <code>struct kapi_win_geom { x, y, w, h; cw, ch; ax, ay, aw, ah; state }</code> — the caller's whole window (frame included), its client size, the work area (the screen less the menu bar at the top and the topmost windows standing on the bottom edge: the dock) → 0 / -1. <code>resize_window2(w, h, &stride)</code>: as <code>resize_window</code>, but the canvas and the frame's copies grow past their first size when needed (new memory at the same addresses; their pixels are lost: redraw, <code>get_chrome</code> again) → the canvas and its stride, 0 (no memory: the size kept). With it: the frame's metrics

Category	Examples
	<code>KAPI_FRAME_TITLE_H</code> 28, <code>KAPI_FRAME_BORDER</code> 4, <code>KAPI_FRAME_RADIUS</code> 8 (the chrome copies' top byte: a transparency, heeded in the corner squares), the title buttons' places <code>KAPI_FRAME_BTN_W/H/Y/EDGE/STEP</code> (the window menu at the left; close, maximise, minimise from the right: <code>KAPI_FRAME_MENU/CLOSE/MAXIMISE/MINIMISE</code>); <code>GUI_EVENT_WINCTL</code> (18: the window menu, maximise — for the app); <code>WIN_FLAG_ALPHA</code> (32: a borderless window's pixels carry their transparency). See §10.2.
Screen size (v66)	<code>screen_set(w, h)</code>: the screen's resolution now (640 × 480 .. 2560 × 1600, <code>w</code> even; <code>ScreenResizeRequest</code>, <code>kernel.cpp</code>). The compositor does it between two frames (the display DMA is idle — each present waits for its end): <code>C2DGraphics::Resize</code> (Circle's: the frame buffer asked of the firmware again at that size, the drawing buffer made again), the old size back if the firmware refuses; then <code>CWindowManager::OnScreenResized</code>: the cursor and every window kept on the screen (moved in when past the right / bottom edge; a window parked at a negative place is left there), the app-written wallpaper dropped (the next <code>wallpaper_buffer</code> is a new buffer of the new size — the old one is left allocated: an app may still have it mapped), and <code>GUI_EVENT_DISPLAY_RESIZE</code> (19, <code>lValue = w << 16 h</code>, <code>GUI_DISPLAY_W/H</code>) to every window's pointer handler. The caller waits for it → 0; −1 out of bounds; −2 not now (a full-screen app owns the display — its mapping of the frame buffer would be stale —, the debug console, another change under way); −3 the firmware refused it (the old size kept). <code>screen_grab</code> refuses the old size (<code>vncd</code> / <code>rdpd</code> see it and take the new

Category	Examples
	one). Not kept across a reboot: <code>cmdline.txt</code> 's <code>width=</code> / <code>height=</code> are (the Display applet writes both). See §10.2.
Workspaces (v65)	The virtual desktops. <code>desk(set, count)</code> : <code>set ≥ 0</code> shows desk <code>set</code> , <code>count > 0</code> sets how many there are (1 .. <code>KAPI_DESK_MAX</code> = 8; the dock keeps 1–6; the windows of the desks dropped go onto the last one); <code>−1 / 0</code> keep them → the current desk`
Held keys (v48)	<code>key_held(key)</code> → 1 while the key is held and the caller's window has the keyboard (<code>KeyTargetLocked</code>), else 0 — for games, since key events only report presses. Keys: <code>KEY_UP/DOWN/LEFT/RIGHT</code> , <code>KEY_ENTER</code> , 27, ' ', 'a' .. 'z' (the US position of the key), '0' .. '9'. The WM keeps two bitsets over the logical codes: the USB one, rebuilt from every raw report (<code>KeyRawStub</code> → <code>SetUsbHeld</code> , HID usage → key), and the injected one (<code>inject_key_held(key, down)</code> , from <code>vncd</code> 's RFB key down / up); <code>KeyHeld</code> ORs them.
FM (v47)	<code>sound_instrument(voice, const struct kapi_fm_instrument *)</code> — a 2-operator FM instrument (OPL2 style, <code>struct kapi_fm_op op[2]</code> = modulator / carrier: <code>mult, level, ksl, attack, decay, sustain, release, wave, flags</code> <code>FM_SUSTAINED</code> / <code>FM_TREMOLO</code> / <code>FM_VIBRATO</code> / <code>FM_KSR</code> ; <code>feedback, connection</code>) for that voice; then <code>sound_start(voice, milliHz, SOUND_FM, volume)</code> . Owner only. See §12.
Memory primitives (v36)	<code>memset</code> , <code>memcpy</code> , <code>memmove</code> — Circle's kernel implementations (general registers only, so callable from any app). <code>user/kapi.h</code> wraps them as weak <code>kapi_memset/kapi_memcpy/kapi_memmove</code> symbols, and the freestanding app Makefiles alias the C names onto them (<code>-Wl,--defsym,memset=kapi_memset, ...</code>): GCC may emit these calls on its own (array/struct initialization, copies) even

Category	Examples
	with <code>-ffreestanding</code> , and freestanding apps have no libc. Newlib programs keep newlib's own.
Crypto (v30)	<code>random</code> (fill a buffer from the Pi's hardware RNG , Circle <code>CBcmRandomNumberGenerator</code> ; for cryptographic seeding — the TLS entropy source in <code>user/tls/onyx_tls.hpp</code> feeds mbedTLS's CTR_DRBG from it)

All the functions **run in the context of the calling app** (its page table + its stack are active; the arguments are plain pointers in the current space — no `copy_from_user`). Some are **modal and synchronous**: `message_box`, `file_open`, `file_save` **block (yield in a loop)** the calling app until the response, while the compositor and the other apps keep running.

Historical note. `ARCHITECTURE.md` §11 describes an earlier approach where the build emitted a `user/kernel_syms.ld` (`kapi_x = 0xADDR;`) and the apps were linked against those addresses. This approach is **no longer used**: the fixed-address table replaced it. The apps no longer depend on the kernel build.

9. Stream / stdio subsystem

Source: `kernel/sys/stream.cpp`, `kernel/include/kern/stream.h`.

This is what makes the terminal's **pipes and redirections** and inter-process communication possible.

Object model

Abstract base class `CStream` with reference counting (`AddRef/Release`; destroyed when the last reference drops). Three implementations:

Class	Role
<code>CPipeStream</code>	in-memory FIFO (ring buffer ~8 KB) between two tasks
<code>CFileStream</code>	FatFs file presented as a stream (read, write/truncate, append)

Virtual methods: `Read` (cooperative blocking: yields until ≥ 1 byte or EOF; 0 = EOF), `ReadNonBlocking` (>0 / 0=EOF / -1=would block), `Write` (may block if full), `CloseWrite` (signals "no more writing" → readers see EOF).

- **CPipeStream**: ring buffer `head/tail`. `Read` yields while empty and the write end is not closed; `Write` yields while full. Cooperative → no lock.
- **CFileStream**: wraps a FatFs `FILE`; modes 0=read, 1=write+truncate, 2=append.

Processes and stdio

- `CAddressSpace` owns `m_pStdin/m_pStdout` (set by `SpawnProcess`).
- `kapi_stdin_read/kapi_stdout_write` read/write these streams (or log if absent).
- `kapi_spawn(path, args, in, out) → SpawnProcess`: the child task takes a reference on `in/out`. `kapi_wait` blocks (cooperatively) on `CProcess::bDone` then returns the exit code. When the child's address space is destroyed, `stdout` gets `CloseWrite` → the reader (the terminal) sees EOF.

The terminal thus chains the `stdout` of one stage to the `stdin` of the next via `CPipeStreams`, and reads the final output non-blocking. App-side details in the [developer guide](#) and the [user guide](#).

10. Graphics subsystem (GUI)

Source: `kernel/gui/{gimage,window,skin,dialog}.cpp` + headers. Rendering core ported from the author's FreeBASIC `SimpleOS`.

10.1 `GImage` — software rendering engine

- **Pixel format: 32-bit 0x00RRGGBB** (alpha byte unused).
- **Transparency color: magenta 0xFF00FF** (transparent-blit key).
- A `GImage` owns its buffer (`SetSize`) or **wraps** one (`Wrap`, e.g. the framebuffer's back buffer or a window's canvas).
- Primitives: `Clear`, `SetPixel/GetPixel`, `DrawLine` (Bresenham), `DrawRectangle`, `FillRectangle`, blits `PutOtherRaw/PutOther` (with key) / `PutOtherPart` (sub-rect, used by the 9-slices skin engine).
- **Text**: Circle's bitmap font (`CCharGenerator`). `FontWidth/Height`, `DrawChar` (transparent background), `DrawText`.
- **BMP**: `LoadBMP` (24 bpp uncompressed, BGR→RGB, handles top-down/bottom-up).

10.2 Windows and compositor

- **CWindow**: position, logical size, title, flags (`WIN_FLAG_BORDERLESS`; `BACKMOST`: the desktop's band; `TOPMOST`: the menu bar, the dock — above every window, never active, never the keys; `TRANSPARENT`: the magenta key; `SYSTEM`: not listed as an open app; `ALPHA` (v64): the canvas's top byte is a transparency), a `GImage` **canvas allocated 64 KB-aligned and physically contiguous** (mapped into the app at `USER_WINDOW_CANVAS` = 12 GB — the app draws directly, with no per-pixel call), the frame's two copies (active, inactive: mapped at `USER_WINDOW_CHROME` / `_INACTIVE`, drawn by the app — `kapi v28 get_chrome`; wtk: `wk_decorate_window`), an event queue (spinlock-protected ring), the app's handlers, its menu.
- **The frame** (v64, the modernised CDE): a 28 px title bar and 4 px borders (`WIN_TITLEBAR_H` / `WIN_BORDER` = `KAPI_FRAME_TITLE_H` / `KAPI_FRAME_BORDER`, shared with the apps in `kapi_abi.h`), rounded corners (radius `KAPI_FRAME_RADIUS` = 8): in the frame's copies a pixel's top byte is its transparency, heeded in the four corner squares only. `DrawTo` blits the frame's **bands** only (the title bar, the bottom border, the two sides beside the client area — never under it) opaque, the corner squares blended by their pixels' transparency (`BlendAlphaRect`), then the client canvas. `CoversOpaque` (`rect`) — may the compositor skip what lies below? — leaves out only the corners' see-through pixels (`CornersIn`: on a corner's row *k* from the edge, its first `CornerSpan` (*k*) pixels — those not wholly inside the arc): a rectangle inside the window still covers. *Borderless* windows have no frame.

- **The present's damage** (`PresentDamage`): an app's present dirties **its client area only** — its whole window when its frame changed since (`get_chrome`, a resize: `ChromeGen`) or it is faded — so a window refreshing alone (an emulator) keeps the compositor's fast path: its client rectangle is covered, nothing below it is drawn. That first present after a frame's redraw also bumps `ChromeGen` once more: the remote desktop (`rdpd`), which may have read the frame while the app was still drawing it (`get_chrome` bumps it **before** the drawing), reads it again, whole — no frame left without its buttons on the PC. A window's two frame copies are allocated both or none (`AllocChrome`: no half-mapped frame).
- **The title buttons** (v64): the window menu at the left; close, maximise, minimise from the right, at `KAPI_FRAME_BTN_*` (the app draws them, the kernel hit-tests them: `HitTitleButton`). Close and minimise act at their release over them (`RequestExit`, `Minimise`); the window menu (at the press) and maximise (also a double click on the title bar) go to the app as `GUI_EVENT_WINCTL` (18; value `KAPI_FRAME_MENU / KAPI_FRAME_MAXIMIZE`): wtk's `Root` shows its window menu, maximises and restores (the developer guide).
- **Minimised windows** (v64, `SetMinimised`): not drawn, not hit, never the active window nor the keys' target (`ActiveLocked`, `KeyTargetLocked`), their area damaged as they go and come back; `Raise` (`win_raise`, `raise_app`: the dock, the menu bar's Open Windows) brings one back.
- **Workspaces** (v65): a window's desk (`CWindow::Desk`, `-1` = every desk: the topmost, backmost and system windows) is the current one when it is added (`CWindowManager::m_nDesk`, of `m_nDesks`, 4 at boot — the dock sets its number from `SD:/etc/dock.ini`). `SetDeskLocked` flags every window of another desk `OffDesk`; `Hidden ()` (minimised or off-desk) is what the compositor, `DrawTo`, `CoversOpaque`, `HitTest`, `ActiveLocked`, `KeyTargetLocked` and the bottom inset heed, and `SetOffDesk` damages the window's area as it goes and comes back. `ForgetHiddenLocked` drops the pointer's state (hover, capture, a drag, a pressed title button) naming a window that went hidden. `RaiseLocked` of an off-desk window shows its desk first. `OnKey` takes **Ctrl+Alt+Left / Right** (the xterm modifier form `ESC[1;7C / D` too) before the apps: the previous / next desk, with Shift the active window moved along (`MoveToDesk`), not while an app is full screen. `DeskInfo`, `SetDesk`, `MoveToDesk` for kapi desk / `win_desk` (each change bumps `m_nDeskGen`: the dock redraws its pager).
- **See-through windows** (`WIN_FLAG_ALPHA`, borderless): each pixel blended by its top byte (0 opaque .. 255 see-through, times the window's fade); never `CoversOpaque`; a click on a wholly see-through pixel goes to the window below (`HitTest` → `OpaqueAt`). The dock (its rounded corners, the gap under it — almost clear, top byte 254, while a drawer is open, to catch a click elsewhere), the menu bar (its drop-downs), the agenda widget (its text straight on the wallpaper).
- **The work area** (`WorkArea`, v64 `win_geometry`): the screen less the top inset (a topmost window at `y = 0`: the menu bar, its smallest height) and the bottom inset (the topmost windows standing on the bottom edge: the dock, its smallest height). New windows are placed in it; a maximised window fills it.
- **A window that grows** (v64 `resize_window2`): `CWindow::Grow` allocates a bigger canvas and frame copies; the process's mappings are moved to them at the same addresses (`MapContig`, then `CAddressSpace::FlushTLB: tlbi aside1is`); the old memory is retired and freed by the compositor three frames later (`FreeRetired`: a frame begun before may still be reading it).

- **The screen's size, while running** (v66 `screen_set`, the Control Panel's Display applet): the compositor task makes the frame buffer again (`C2DGraphics::Resize`) between two frames, then `OnScreenResized` keeps the windows on the screen and sends them `GUI_EVENT_DISPLAY_RESIZE`. The apps placed by the screen's size place themselves again on it: the menu bar (its canvas grown to the screen: `resize_window2`), the dock (`onDisplayResize`: laid out for the new width, along the bottom), the notifications (the top right corner); wtk re-maximises a maximised window and shrinks / moves one past the work area ~0.3 s later (`Root::displayTick`: the dock has moved, the work area is the new one). vncd sends the VNC `DesktopSize` pseudo-rectangle (-223) to a client that takes it (else it closes the session: the client connects again at the new size); rdpd sends `SCREEN` (8: u16 w h) and Onyx Remote resizes its view of the Pi's screen.
- **CWindowManager** (singleton): a Z-ordered list (bands: backmost, normal, topmost; the last = on top), protected by a `CSpinLock`. `Add/Remove/Raise/Minimise`. `Composite(screen)`:
 1. snapshot the list under the lock (the compositor also frees the grown windows' old memory), then blit outside the lock;
 2. find the topmost window that covers the clip rectangle opaquely: nothing below it is drawn;
 3. otherwise clear the desktop and blit the wallpaper;
 4. draw the windows back to front (`DrawTo`, `bActive` for the keys' window);
 5. the drag badge, then the cursor (`mousecur.bin` bitmap or a fallback arrow).
- **Dirty rectangles**: `Composite` honours the screen image's clip rectangle, so the compositor task redraws only the damaged rectangles (see the service tasks above); host tests `sh tools/tests/run_gui_test.sh` (clipped drawing = full drawing inside the clip, nothing touched outside) and `sh tools/tests/desktop_sim/run.sh` (the v64 / v65 windows: `wmtest.cpp` — the corners, `CoversOpaque`, the present's damage, the title buttons, minimise, the see-through windows' clicks, the work area, a canvas growing, the workspaces).

10.3 Widgets (kernel side)

Types: button, label, checkbox, text box (1 line), progress bar, slider, multi-line text area, V/H scrollbar, icon (image + label, with an open-app "badge"). Each widget stores the app's **callback address**. The window manager does the hit-test, the rendering and **pushes the event**; the app's `pump_events` dispatches it **in the app's context**. Events:

Constant	Meaning
<code>GUI_EVENT_CLICK</code>	button/icon released over it
<code>GUI_EVENT_CHECK_CHANGED</code>	checkbox toggled
<code>GUI_EVENT_TEXT_CHANGED</code>	textbox/textarea modified
<code>GUI_EVENT_VALUE_CHANGED</code>	slider/scrollbar moved (0..100)
<code>GUI_EVENT_KEY</code>	key pressed (<code>value</code> = char or <code>KEY_*</code>)
<code>GUI_EVENT_CANVAS_CLICK</code>	client click with no widget: `value=(buttons < 32)
<code>GUI_EVENT_CANVAS_MOTION</code>	drag with button held (same coords)

Mouse handling: hover tracking, press edge (raises the window, hit-test of the close box / title bar / widget / canvas), drag (window move or continuous slider/scrollbar), release edge (click/toggle). Focus

follows the click (textbox/textarea). The keyboard (Circle's "cooked" VT100 strings) is translated into logical keys (`KEY_UP`, `KEY_ENTER`, ...) and delivered to the modal dialog, otherwise to the focused widget, otherwise to the app's keyboard handler.

10.4 Theme, wallpaper

- **The theme** is the apps' business: the windows' frames, like every control, are drawn by the apps (wtk: `user/wtk/skin.cpp`, `paint.cpp` — by code, no bitmap) from `SD:/etc/theme.txt` (`theme = Peach / Steel / Sage / Brick / Slate`, or `active; inactive; face; accent; outline; dock`: read by `wtk/theme.cpp`); the kernel only blits the frames. Of that file the kernel reads only `wheelsspeed=N` at boot. (The old 9-slice window skin — `wings.bmp` tinted by `CSkin` — and `kapi_set_window_theme` are no longer used by the desktop.)
- **Wallpaper**: `set_wallpaper` (BMP), `wallpaper_generate` (toroidal Voronoi generated at runtime), or **an app-drawn background**: `wallpaper_buffer` maps the screen-sized shared buffer at `USER_WALLPAPER_CANVAS` (13 GB), the app draws, `wallpaper_commit` makes it live. The frames are **owned by the kernel** → the background persists after the app exits (the `voronoy` case). The agenda widget reads it to choose its ink.

10.5 Modal dialogs

`CDialog` (types: `DLG_MSGBOX`, `DLG_FOPEN`, `DLG_FSAVE`). The call runs in the kernel (not preempted), so the calling app **yields in a loop** in the kernel as long as the dialog is not resolved; meanwhile **the compositor runs** and draws the dialog **on top of** the owner window (blocked), and the other apps stay usable. The file dialog lists a FatFs directory (folders first, `..` to go up), with keyboard selection and, for `FSAVE`, an editable name field.

11. Network subsystem (WLAN, TCP/IP, NTP)

Sources: `kernel/kernel.cpp` (`CNetBringupTask`), `kernel/sys/net.cpp` (socket backend), `kernel/include/kern/net.h`. Built on Circle's `lib/net` (TCP/IP) + the `addon/wlan` BCM4343 driver + `wpa_supplicant`, all linked into the kernel (see `kernel/Makefile LIBS`).

Two placements, chosen at boot by `cmdline.txt netcore=`:

- `netcore=0` (the default): the whole stack on the primary core, as ordinary cooperative tasks among the others; the socket calls run on the app's task.
- `netcore=1`: **the network core**. Core 3 runs the stack with its own scheduler (§5): at boot it waits in `WFE` (`NetCoreMain`, from `COnyxCores::Run`); once the SD card is mounted, core 0 calls `NetCoreStart`, and core 3 creates its `CScheduler`, the bring-up task (below) and 6 **worker** tasks. Everything the bring-up starts — Circle's `CNetTask`, DHCP, NTP, the WLAN driver's `kprocs`, `wpa_supplicant` — is created on core 3 and runs there, so the stack is still used from one core only, as it was written for. The WLAN firmware and `wpa_supplicant.conf` are read from core 3 through FatFs: the volume lock (`sys/fslock.cpp`) takes its owner with an atomic compare-and-swap. The SDIO interrupt stays on core 0 (its handler only masks the controller's interrupt enable; the driver polls), and so do the TCP / ARP kernel timers (they set flags under spin locks, which the stack reads in `Process`).

- **Requests.** A socket kapi (`tcp_*`, `net_resolve`, `net_ping`, `net_info`, `wlan_scan`) copies its arguments — the app's memory is not mapped on core 3 — into one of 32 request slots (8 KB of data each: a bigger send goes in pieces), marks it *posted*, and waits: it spins 300 μ s (most answers take microseconds), then yields, then sleeps 1 ms, then 10 ms steps (an `accept` may wait for hours). A worker claims the slot (compare-and-swap), runs the same `Do*` function as `netcore=0`, and marks it *done*; a `recv` gathers several segments (up to 8 KB) in one round trip. When every worker waits (accepts, connects), the core adds one (up to 24). `net_status` reads the state directly.
- **A process that dies** (in a request, or with sockets open): `NetCloseByPid` (its teardown, IRQs masked: nothing waits) drops its posted requests, orphans the running ones (the worker then closes what they opened), and queues its pid in a ring that the net core's main loop reads to close its sockets.
- **Notices.** `IpcNotify` is core 0's: the bring-up leaves its "Connected" notice to `NetCoreNotify`, and the kernel's main task delivers it (`NetCorePoll`, every 250 ms).
- Core 3 is then no longer an app core (§14).
- **Bring-up is non-blocking and non-fatal.** `CNetBringupTask` (a kernel `CTask`, started from `Run()` once the SD card is mounted) runs `CBcm4343Device::Initialize` → `CNetSubSystem::Initialize(FALSE)` → `CWPASupplicant::Initialize`, waits for the DHCP bind, logs `net: up, IP ...`, then **returns** (the task ends cleanly). A missing firmware / `wpa_supplicant.conf` / access point only logs a warning — the GUI boots regardless.
- **Self-driving.** `CNetSubSystem::Initialize` spawns Circle's own `CNetTask` / `CPHYTask`; they run as ordinary cooperative tasks on our scheduler (§5), so no explicit pumping is needed. (This is also why the net task is CPU-busy — the reason for `netcore=1`.)
- **Globals.** `g_pNet` (the `CNetSubSystem`) and `g_bNetUp` are published in `net.h`; `NetIsUp()` gates the socket calls.
- **Sockets.** `sys/net.cpp` keeps a small table of Circle `CSockets` behind integer handles. `tcp_connect` resolves a dotted-quad or a DNS name (`CDNSClient`) and connects (blocking, cooperative); `tcp_send` blocks with a 5 s timeout; `tcp_recv` is **non-blocking** (so a GUI app polls it from its frame loop); `tcp_close` drops it. Each socket records its **owner pid**, and `AddressSpaceTaskTerminate` calls `NetCloseByPid` so a process that dies without closing does not leak its connections or table slots.
- **Clock.** Once the link is up the bring-up task starts a `CNTPDaemon` (`system.ini ntp=`), which updates `CTimer`'s wall clock; the boot reads `system.ini timezone=` (minutes from UTC) and calls `CTimer::SetTimeZone` so `get_datetime`, the agenda and the log timestamps show local time.
- **Caveats.** Plain-text only (no TLS); `MAX_TASKS` was raised to 40 to fit the net workers; the firmware load uses FatFs and is not locked against concurrent app file I/O (low risk, one-shot at boot) — with `netcore=1` it is (the atomic volume lock).

The apps that use it: the **irc** client (`user/irc.c`) and the **net** /`bin` tool (link status / IP).

12. Sound and the second core

Source: `kernel/sys/sound.cpp`, `kern/sound.h`.

- **Multi-core.** Circle is built with `ARM_ALLOW_MULTI_CORE` (fork patch #4, [Circle Changes](#)). `CKernel::Initialize` starts cores 1–3 through a `CMultiCoreSupport` subclass (`COnyxCores`, `kernel.cpp`) right after the kapi table is published. **Everything else stays on core 0:** the scheduler, every process, the interrupts (the GIC routes peripherals to core 0; core 1 runs Circle's own `VectorTable`, not our `KVectorTable`). Core 1 runs `SoundCoreMain`; cores 2 and 3 are **app cores** (§14). A failed start is only a warning (no sound producer, no app cores).
- **The device.** `COnyxSoundDevice` derives from Circle's `CPWMSoundBaseDevice` (PWM + DMA, the 3.5 mm jack; 44.1 kHz, 1024-frame chunks) and overrides `GetChunk` — the "producer" — which is called from the DMA completion interrupt. It is created and started on the first `sound_acquire` (not at boot).
- **Core 1 = the producer.** It sleeps in `WFE` until the device starts, then keeps `SND_AHEAD` (4) chunks rendered ahead in a ring; `GetChunk` only copies the next chunk (zeros if core 1 fell behind) and `SEVs` core 1. Without `ARM_ALLOW_MULTI_CORE` the same code renders in `GetChunk` itself (interrupt, core 0).
- **Rendering** (integer only, no FP in the kernel): 16 voices, each a 32-bit phase accumulator (`inc = milliHz · 232 / (1000 · 44100)`), waveforms square / sine (256-entry table, linear interpolation) / triangle / saw / noise (LFSR per period), a linear ~5 ms attack/release envelope (no clicks), mixed ÷4 and clipped; plus the **PCM ring** (0.5 s of s16 stereo frames, `sound_write`). Converted to the PWM range per sample.
- **FM voices (v47).** `sound_instrument` (`voice`, `struct kapi_fm_instrument`) gives a voice a 2-operator FM instrument in the style of the AdLib's OPL2 (op 0 = modulator, op 1 = carrier): multiplier, output level (0.75 dB steps), attack / decay / sustain level / release rates (OPL2 timings: attack 2.8 s at rate 1, decay / release 39 s over 96 dB at rate 1, halving per step), sustained (EG type), tremolo (1 dB, 3.7 Hz) / vibrato (7 cents, 6.1 Hz), waves sine / half / absolute / quarter pulses, feedback 0–7 and connection (FM or additive). Then `sound_start` (`voice`, `f`, `SOUND_FM`, `volume`) keys it on (both envelopes restart) and `sound_stop` keys it off (the release rate fades it). The envelopes run on an attenuation in 1/256 octave units (4096 = 96 dB), turned into amplitude by a 256-entry $2^{(-x/256)}$ table; the modulator bends the carrier's phase by up to ±4 periods (as OPL2). Tables: `sys/sound_tables.h` (generated). The same file builds on a PC with `SOUND_HOST_TEST` (the Circle parts left out): `tools/tests/run_fms_test.sh` renders instruments with it, and the Windows FM Song player (`tools/fmsplayer`) plays with it.
- **Low latency (v68, `sound_config`).** `CPWMSoundBaseDevice`'s chunk size is fixed when it is made — but it is only the size of its two DMA buffers: Circle's `CDMASoundBuffers` programs each transfer with the length `GetChunk` returns. So the device is made once, with the biggest chunk (1024 frames), and a smaller chunk is just a shorter return: nothing re-created, the PWM clock untouched, no Circle change. The owner picks the chunk (`s_nChunkFrames`, 64 .. 1024 frames) and how many chunks core 1 keeps ahead (`s_nAheadCfg`, 1 .. `SND_AHEAD` 4); each slot of the ahead ring records its own length (a change applies from the next chunk rendered — the ones already rendered play out first). What is heard lags the rendering by about **(ahead + 1) chunks** (the ahead ring + the DMA buffer queued behind the one playing): 1024×4 (the default, what the emulators use) $\approx$ 116 ms; $256 \times 2 \approx$ 17 ms; $128 \times 2 \approx$ 9 ms; $64 \times 1 \approx$ 3 ms (a DMA interrupt every 1.5 ms; core 1 must never be late: with 1 chunk ahead any hiccup is a click). Add what the app keeps queued in the stream or the ring. `GetChunk` now frees a slot only after copying it (it used to free it first: with 4 slots core 1 could render into the one being copied). `sound_release` / the owner's death restore 1024×4 .
- **The mapped ring (v68, `sound_map`).** One 64 KB page (`SoundRing`: a 64 KB-aligned heap block, made on first use and **kept for ever** — core 1 may be reading it when its owner dies) holding

`struct kapi_sound_ring`: a header (`wr` the app's, `rd` the kernel's — free-running frame counters —, `dry`, the rate, the chunk / ahead now) and 8192 s16 stereo frames (~186 ms). `kapi_sound_map` maps it into the owner like a shared surface (`CAddressSpace::MapSurface`, the surface arena; the teardown drops the mapping only), once per process. `Render` reads `wr` once per chunk, mixes up to a chunk of frames with the voices and the stream, then moves `rd` (DMB in between both sides); `dry` counts the times the ring ran out while playing. The kernel only ever indexes it masked by its own capacity and treats `wr - rd > 8192` as empty, so nothing an app writes there can make it read elsewhere. Since it is plain memory, **an app core fills it** (`kapi_sound_ring_write`), and a core-0 thread can sleep on `rd` with `wait_word` (the tick sees core 1 move it) — the DAW's engine writes straight into the kernel's ring, no pump thread in the audio path. Apps run at EL1 without isolation from the kernel, so a former owner that kept its mapping could still write into the page; the kernel stops mixing it at the release. `ringtest` plays a tone from an app core this way.

- **Ownership.** One pid owns the output (`sound_acquire`); every other call from another pid returns `-1`. `sound_release`, or the owner's exit (`SoundOnProcessGone`, called from `IpcOnProcessGone`), silences the voices, empties the ring and frees the output. The state is shared between core 0 (`kapi` calls) and core 1 (rendering) under a `CSpinLock`.
- **kapi v46:** `sound_acquire`, `sound_release`, `sound_start` (`voice`, `milliHz`, `wave`, `volume`), `sound_stop` (`voice` | `-1`), `sound_write` (`frames`, `n`), `sound_status`. Users: `/bin/tone`, `BASIC PLAY / SOUND / BEEP / NOTEON / NOTEOFF`.
- **kapi v68:** `sound_config` (`chunk_frames`, `ahead`), `sound_map` (`()`) (above).

13. Post-mortem debug console

Source: `kernel/sys/debugcon.cpp`, `kernel/include/kern/debugcon.h`.

Purpose: when an app terminates (or in case of trouble), make the logger's last messages visible **directly on the framebuffer**.

- **CFbConsole:** text console that renders the characters (green on black) on the back buffer of `C2DGraphics`, with line wrapping, scrolling, and `UpdateDisplay()` after each write.
- **CLogSwitch:** routes the logger's output either to the boot console (`m_Screen`), or to the framebuffer console. At boot: `SetNormal(&m_Screen) + DebugConsoleRegister`.
- Once the compositor is started, `MuteNormal()` stops the writes to the boot console: it is no longer shown, and each line drawn there (a text-mode scroll of the whole screen inside `CTerminalDevice::Write`, IRQs masked) froze everything for ~170 ms — an app that `printfs` without a terminal (Doom at start-up) froze the GUI and the network for seconds. `kmsg` still sees every line (`CLogger`'s event ring does not depend on the target).
- When the debug console takes over (`DebugConsoleTakeover`, called on `exit`), the compositor **detects** `DebugConsoleActive()` and **stops presenting** so as not to contend for the framebuffer.

The crash record (`lastcrash.txt`)

Source: `kernel/sys/crashlog.cpp`, `kern/crashlog.h`.

A frozen Pi (the scheduler stopped: the green LED stays lit or dark, no SOS) left nothing to read. Now 64 KB of RAM kept out of the heap (Circle fork, docs/05 §15) hold, for the running session, cleaned to RAM as they are written:

- the **tail of the kernel log** (`CLogSwitch::Write` copies every line, ~62 KB ring);

- the **last 16 places core 0 was interrupted at** (`KernelIRQExit`, each IRQ): PC, LR, SP_ELO, whether IRQs were masked, the task's name — a kernel loop shows itself there;
- **breadcrumbs**: the GPU (`idle` / `clipping` / `binning` / `rendering` / `texture upload`) and the compositor's display copy (`composing` / `display DMA` / `waiting for the display DMA`);
- the reaper's **last pass** (uptime), and a **panic**'s registers (`DumpAndHalt`, before the panic screen) with the **return addresses found on the faulting context's stack** (`CrashLogStack`);
- the **free memory**, once a second (`CrashLogMemory`, breadcrumbs 5–7): the kernel heap, the kernel pages (low pager), the app pages (the high zone) — an "Out of memory" shows its approach;
- the **app the GUI watchdog finds frozen** (below): its pid, the last 16 places its task was interrupted at, and its stack's return addresses.

Return addresses. The apps are built `-O2` (no frame pointers to follow): `ScanStack` reads 16 KB up from the context's SP and keeps the words that point just after a `BL` / `BLR`, in an app's code (`[8 GB, 10 GB)`) or the kernel's (`[0x80000, _etext)`) — 24 at most, newest first. Each page is probed first with `AT S1E1R` (a wild SP, an unmapped page: no fault). Read them with `aarch64-none-elf-addr2line -f -C -e user/<app>.elf <addr>` (or `kernel/kernel18-rpi4.elf`); a few are stale words, the first ones are the calls in progress.

The **BCM watchdog** is armed by the reaper (every second, `cmdline.txt hangreboot=` seconds, default 15, at most ~15; 0 = off): when the scheduler stops, the Pi restarts by itself (a panic too, after its screen and SOS). `kapi_shutdown` / `kapi_reboot` stop it and mark the session clean. At the next boot, a session that did not end cleanly (its record still in RAM: a power cut loses it) is written to `SD:/etc/lastcrash.txt` and noted in `kmsg` (`crashlog`). Best effort: nothing if the RAM did not keep its contents through the reset.

Core 1 writes the report itself (the RAM does not survive the Pi 4's watchdog reset: the first real freeze left no record). At boot (`CrashLogReport`, SD: mounted) the kernel keeps the previous dump if `SD:/etc/crashdump.txt` holds one (`ONYX CRASH REPORT ... --- end of the report ---` → `SD:/etc/lastcrash.txt`), then fills that 64 KB file with "armed" and notes each of its 128 sectors' LBA (through FatFs: `f_lseek` + a one-byte `f_read` → `FIL.sect`). Core 1, the sound core (`SoundCoreMain`), is woken at least every ~1.2 ms by the generic timer's event stream (`CNTKCTL_EL1.EVNTEN`, `EVNTI 15`) and calls `CrashLogCoreCheck`: when the reaper's `CNTPCT` stamp has not moved for **10 s**, it formats the record (no heap, no logger), writes it **raw** through the `emmc1` device into those sectors (runs of consecutive sectors in one write; `g_bCrashDumping` keeps `OnyxDriverWait` from yielding) and restarts the Pi (`CBcmWatchdog::Restart`). No FatFs, heap, logger or scheduler call: core 0 may hold their locks; core 1 takes the sound lock only with `TryAcquire`, so a core 0 frozen while holding it does not stop the watch. Its progress is kept in the RAM record (`nDumpStep`: started, text ready, written, the write failed): when the SD write cannot finish (a wedged bus), the hardware watchdog restarts the Pi and the RAM report (the record of an 8 GB Pi sits at the top of the RAM above 4 GB) says how far core 1 got. **A stuck GUI with the scheduler alive** (the compositor without a frame for 12 s, the GUI watchdog task) asks for the same report (`CrashLogRequest`: core 0 masks its IRQs and waits, core 1 writes it, then the restart). **A Circle panic** (an assertion, the kernel heap's "Out of memory", ...) used to leave nothing: Circle's logger halts **every** core after the panic line (`CMultiCoreSupport::HaltAll`), core 1 too, so the Pi stayed frozen ~15 s and the hardware watchdog restarted it without a report. The logger's panic handler is now `CrashLogPanicHandler`: the log's last line (the panic's) goes into the record and core 1 writes the report at once (`CrashLogDumpNow`), before the halt. An exception (`DumpAndHalt`) asks for it at once too, after its screen and one SOS (it used to wait the 10 s of a stopped reaper).

The app watchdog (`SD:/etc/apphang.txt`). When the GUI watchdog finds an app not taking its window's events (2 s, some queued), it watches that app (`CrashLogWatchPid`: core 0's IRQs note where its

task is and, every 8th time, scan its stack) and, every 2 s while it lasts, has the `apphang` task (its own: never the GUI watchdog stuck on the card) rewrite `SD:/etc/apphang.txt`: the app, its tasks' states (running / ready / blocked / sleeping) and pages, then the whole record (the free memory, core 0's samples, the app's samples and stack) and the log's last 12 KB. The compositor without a frame does the same (every 2 s, before the 12 s report). Recovered (the app takes its events again, or is closed / killed): renamed `SD:/etc/lasthang.txt`; a clean shutdown / reboot renames it too. Still there at the next boot (the session never ended cleanly — the freeze grew into a restart): appended to `SD:/etc/lastcrash.txt` (`MergeAppHang`), or written as it when the kernel kept no record of its own. **Power and heat.** Once a second the GUI watchdog task calls `CrashLogPower`: the firmware's `GET_THROTTLED` bits (under-voltage, ARM frequency capped, throttled, soft temperature limit — now, or since boot) and the SoC temperature. A change (or each 5 °C above 60 °C) is logged (`power: SoC 71 C, throttling: under-voltage (earlier)`), and the last values plus the session's highest temperature go into the crash record (breadcrumbs 2–4), printed in every report — a freeze that only a cold boot brings back smells of the supply or the heat.

The clock across boots. The Pi has no battery-backed clock: until NTP answers (~15 s), the time was 0 and the files written meanwhile (`crashdump.txt`, `lastcrash.txt`) had no date. `SD:/etc/clock` keeps the last time seen (UTC seconds, text; written every 10 minutes by the GUI watchdog task and by `kapi_shutdown` / `kapi_reboot`) and is read back at boot, right after SD: is mounted (`CrashLogClockRestore`) — behind by how long the Pi was off, until NTP corrects it.

Headless signs on the green ACT LED (a GPIO set / clear, no lock): **3 s of fast blinks** when core 1 sees the hang, the SD write's own flicker, then **3 s lit** (written) or **3 slow blinks** (the write failed) before the restart; no blinking at all = core 1 stuck too (a wedged bus). `hangreboot=0` turns it off with the watchdog. `kmsg` tells at boot where the record is and whether the dump is armed (`crashlog: ...`). Test: `hangtest` (IRQs masked) / `hangtest irq`.

The panic screen is now copied into the displayed frame buffer **by the CPU** (not the DMA: the compositor's display DMA may be in flight, and waiting for it hung the panic before its screen and its SOS).

14. App cores (cores 2 and 3)

Source: `kernel/sys/appcore.cpp`, `kern/appcore.h`; user side `user/emucore.h`, test `user/bin/coretest.c`.

Cores 2 and 3 (core 2 only with `netcore=1`: core 3 then runs the network, §11) are a **resource an app acquires**, like the sound output: it gets a whole core and runs one function of its own code there, undisturbed (no scheduler, no timer, no other task on that core). The rest of the system stays on core 0 as before, so none of the kernel, Circle's drivers, FatFs or the network has to be multi-core safe.

- **The core's loop** (`AppCoreMain` → `AppCoreLoop`, from `COnyxCores::Run`): it installs **our** `KVectorTable` on that core (`VBAR_EL1` is per core), notes its kernel stack, enables IRQs (for the stop IPI only: the GIC routes no peripheral there) and waits in `WFE`. A job: load the owner's `TTBR0` (its L2 table + ASID, `CAddressSpace::GetTTBR0`), `tlbi vmal1e` on that core only (an ASID may have been reused since its last job), then `AppCoreCall` (`fn`, `arg`, `stack`) — at `EL1t`, `SP` = the app's stack, like the app itself on core 0. When `fn` returns: back to the kernel address space, state `IDLE`, `SEV`.

- **The kapi side (core 0).** `core_acquire` hands a free, started core to the caller's address space; `core_run` checks the owner, that nothing runs, that `fn` and the stack are user addresses, fills the job, then `bGo + DSB + SEV`. Every word shared between the cores is plain cacheable memory (inner-shareable, coherent) with `DSB ISH` barriers; only core 0 writes the ownership.
- **Stopping a job** (`core_release`, the app's exit or kill): core 0 raises `bAbort` and sends the core an IPI (`SendIPI, IPI_USER`). The IRQ enters our `IrqEntry` on that core; `KernelIRQExit` sees it is not core 0 (no scheduling there) and calls `AppCoreOnIRQExit`, which **rewrites the trap frame** to return into `AppCoreRestart` on the core's own kernel stack (EL1t, IRQs masked): the job is simply dropped. `AppCoreRestart` goes back to the kernel address space and clears `bAbort` — core 0's signal that the core is out of the app's memory. Core 0 waits for it at most 200 ms.
- **Faults.** A synchronous exception on core 2–3 (a bad access in the job) reaches `SyncHandlerEL1`, which hands it to `AppCoreOnFault` instead of the kernel panic: the ESR class, PC and fault address are kept, the state becomes `FAULT` and the frame is rewritten to `AppCoreRestart` as above. Core 0 logs it once (`appcore: core N: fault EC=... at pc ... (address ...)`) when the owner asks the state or releases the core.
- **Teardown.** `~CAddressSpace` calls `AppCoreReleaseAS (this)` **before** freeing the window and the frames: a job still running uses them. A core that does not answer the stop (its code masked the interrupts — never do that) is **retired** (`bLost`, never used again) and the dying space's memory is kept (leaked) rather than freed under it.
- **The rules for `fn`:** no kapi call (the kernel is not called from two cores at once), no allocation (newlib's and umm's `malloc` are not multi-core safe), no masking of the interrupts. It computes, reads the clock directly (`cntpct_el0`) and exchanges data with the app's main thread through memory, with barriers. It may use `WFE`, woken by the main thread's `SEV`.
- **`user/emucore.h` packages** that for the emulators: the machine runs on the app core (`ec_thread`), the main thread asks for frames (`ec_request`), gets the pictures from a **triple buffer** (`ec_publish / ec_take`: the machine never waits for the display and the display always gets the latest complete picture) and the sound from a single-producer ring (`ec_audio_push / ec_audio_pop`), and stops the machine between two frames to touch it (`ec_hold / ec_resume`: reset, palettes, battery saves). Without a free core, `ec_pump` runs the same frames on the main thread. `gbemu`, `gbaemu`, `nesemu` and `snesemu` use it.
- **Test:** `/bin/coretest` (the same computation on an app core and on core 0, a job stopped by its flag, an endless job stopped by `core_release`, a faulting job, both app cores at once); `coretest exit` leaves a job spinning and exits (the teardown must stop it).
- **Bigger programs on an app core:** newlib's syscalls can run on the main thread when called from an app core (`libc/onyx_syscalls.c`, `onyx_rpc_*`: the caller posts the call and waits in `WFE`, the main thread runs it in `onyx_rpc_serve` and `SEVs`). Doom's engine runs that way (`user/doom`), with its `malloc`, files and saves.

15. The GPU (V3D)

Source: `kernel/sys/v3d.cpp`, the packets `kern/v3d_c1.h`, the shaders `sys/v3d_shaders.qasm`, the texture layouts `kern/v3d_tiling.h`; demos `user/Apps/teapot (v52)`, `user/Apps/gpudemo (v53)`.

The Raspberry Pi 4's VideoCore VI 3D core (**V3D 4.2**) driven from the kernel, bare metal — after Random06457's *rpi4-gpu-bare-metal-examples* (MIT: the packet structs, the three shaders, the control-list recipe) and macoy's *rpi-system* notes (cache cleaning, the biner's memory).

- **Bring-up** (the first `gpu_info` / `gpu_draw`, never at boot): the firmware turns the V3D power domain (10) and clock (5) on and the clock to its maximum (mailbox); the reset is released (`PM_GRAFX V3DRSTN`, with the PM password) and the V3D's async AXI bridges opened (`ASB_V3D_M_CTRL` / `_S_CTRL` at `0xFEC11008/C`, waiting for the ACK bit with a time limit); then `HUB_IDENT1` must say version 4 (kmsg: `HUB_IDENT0..2, CTL_IDENT0`). Any failure leaves the GPU off; `gpu_info` says why. The interrupt is connected there too (see below).
- **Memory**: no V3D MMU — the GPU gets physical addresses, all its buffers come from Circle's low heap (`HEAP_LOW`, < 1 GB, page aligned): the control lists, the shaders and default attributes, the vertices, the render target (raster RGBA8), the tile allocation (the tiles × 64 B + 2 MB), the tile state (256 B a tile) and a 4 MB overflow pool. CPU caches: cleaned after the CPU writes (the lists, the vertices), the target cleaned and invalidated before the GPU writes it and invalidated before the CPU reads it; the GPU's L2T and slice caches are flushed before each job.
- **A frame** (`gpu_draw`): the vertices are copied into the GPU buffer; the **binning list** (`TILE_BINNING_MODE_CFG`, the clip window, `CFG_BITS` with the depth test *less* and depth writes, the viewport scaling and offset, `GL_SHADER_STATE` → a shader record + two attribute records over the same 16-byte vertices — position 3 floats, colour 4 normalized bytes —, `VERTEX_ARRAY_PRIMS` triangles) sorts the triangles into 64 × 64 tile lists; the **rendering list** configures the tile buffer (one RGBA8 target, Z cleared to 1.0), clears, then for each tile runs its list, stores the colours (`STORE_TILE_BUFFER_GENERAL`, raster) and clears again. Jobs: `CT0QMA/QMS/QTS/QBA/QEA` (bin), `CT1QBA/QEA` (render); the end is polled on the frame counters `BFC` / `RFC` (the caller's task yields meanwhile; one frame at a time), 0.5 s at most — past that the state of the control-list executors is logged and the GPU left off. The binner's out-of-memory requests (`INT_STS OUTOMEM`) are answered from the overflow pool (`PTB_BPOA` / `BPOS`). The RGBA8 picture is copied into the caller's pixels as `0x00RRGGBB`.
- **Shaders**: fixed — a vertex shader (position → screen, colour → varyings), its coordinate shader for the binner, a fragment shader writing the interpolated colour: Mesa-compiled QPU code taken from Random06457's example. The geometry, the lighting and the projection are the app's (the CPU); the GPU does the rasterization, the interpolation and the depth test.
- **The full pipeline** (`kapi v53`, `gpu_render`): the same frame recipe with **batches**. Each batch has its own shader record (the attributes: position x y z w — 4 floats —, s t, colour; s t, colour, added colour; 32-byte vertices), its uniforms (the batch's 4 × 4 matrix, then the viewport scales), its `CFG_BITS` (depth function and writes, culling: `clockwise_primitives` set, front = counter-clockwise in y-up coordinates, as GL), its blending (`BLEND_ENABLES` + `BLEND_CFG`: alpha `SRC_ALPHA` / `INV_SRC_ALPHA`, add `SRC_ALPHA` / `ONE`, multiply `DST_COLOR` / `ZERO`, premultiplied `ONE` / `INV_SRC_ALPHA`) and one `VERTEX_ARRAY_PRIMS` over its range of the vertex buffer. `KAPI_GPU_F_KEEP` loads each tile from the target first (`LOAD_TILE_BUFFER_GENERAL`, raster RGBA8, alpha forced to 1) instead of clearing it.
- **Shaders of v53** (hand-written: `sys/v3d_shaders.qasm` → `v3d_shaders.inc`, assembled by `tools/qpu/qpuasm`, see below): `VS_CLIP` — the 10 inputs, the matrix product (16 `ldunif`), 1/w on the SFU (`recip`, `r4`), `Xs Ys` in 24.8 fixed point, `Zs`, 1/Wc, then the 10 varyings `s t r g b a r2 g2 b2 a2` (v54); `CS_CLIP` — its binner copy (clip `X Y Z W`, then `Xs Ys`); `FS_COLOR` — the interpolated colour (varying = `ldvary` × W + C, C in `r5` two instructions later); `FS_TEX` — the texture lookup (T to `tmult`, the TMU configuration `p0` / `p1` by `wrtmuc`, S to `tmus` starts it, as Mesa orders them; a thread switch while it runs; `ldtmu` returns R G and B A as half floats) multiplied by the colour; both then add the second colour and clamp to 1 (v54). `FS_COLOR_AT` / `FS_TEX_AT` (v54, `B_ALPHATEST`) read the threshold as one more uniform and discard the fragment (`fsub.pushn`, then `setmsf.ifa -, 0`)

when alpha is below it. The fragment shaders end with the TLB writes (`vfpack tlb`) after the last-segment thread switch.

- **App shaders** (`kapi v61`, `gpu_program / gpu_render2`): the app brings its QPU code (built at run time with the C++ builder `user/v3d/qpu.h` over Mesa's instruction packer, `tools/qpu/mesa`; tested on the PC by the simulator `tools/qpu/qpusim` — `tools/tests/run_qpu_test.sh`). A program (`TProgram`, ≤ 256 , owned by its process) holds the three shaders in low GPU memory, each at a 256-byte boundary and followed by NOPs (the QPU fetches ahead). Each batch of `gpu_render2` gets its shader record from the program: the VPM segment sizes ($\lceil \text{inputs} / 8 \rceil$, $\lceil (4 + \text{varyings}) / 8 \rceil$, the coordinate shader's $\lceil \text{csInputs} / 8 \rceil$ and $\lceil \text{csOutputs} / 8 \rceil$), the fragment shader's threading (`P_FS_4WAY`, `P_FS_FINAL`) and Z writes; one float vec4 attribute per 4 inputs (≤ 16 arrays: the `GL_SHADER_STATE` packet is written by hand, its count has 5 bits), its uniforms copied from the call's array — the samplers first (32-byte aligned), then the vertex, coordinate and fragment ranges, the texture words patched in the latter —, then `CFG_BITS`, `BLEND_CFG` (the batch's own factors and equations), `COLOR_WRITE_MASKS` and `CLIP_WINDOW` (the scissor) when they change. The kernel does not check what the shaders do: a program that never ends is stopped by the frame's time limit (the GPU is then left off), and the TMU's general writes could reach any low memory — apps are trusted, as with `code_alloc`. **P_FS_FINAL is best avoided**: a fragment shader started in its final thread section (no last-segment pair) that reads uniforms stopped the GPU now and then on targets 512 pixels wide or more (5 runs out of 7, mostly the first frame after boot; 64×64 frames always passed; `v3dprog ww`, docs/03). Mesa never uses it for fragment shaders; neither do `user/v3d/gxtev` nor `gcemu` since: they end with the last-segment pair (two `thrs` in a row, the live values in registers of the register file — the accumulators do not survive a thread switch) before the TLB writes.
- **gpu_vbuf / gpu_render3 in place** (`v63`, `ClipInPlace`): a vbuf is a `HEAP_LOW` block (64 KB aligned, below 1 GB: its address is the GPU's) mapped into the program by `MapSurface` (`s_vBuf`, freed by `V3DReleaseAS`). When `gpu_render3`'s array lies in one of the caller's, each batch's vertices are framed where they are and tested; the triangles inside every plane make runs `{first, count}` of the batch's own vertices (`s_pRun`, `s_pRunAt`: `Render2` draws each run with the batch's shader record, its attributes at the batch's address and stride), the others are clipped into the buffer's free end, each fan at a whole number of the batch's strides from it (a run too); then the touched range's cache is cleaned. The copying paths (`ClipFrame2 / 3`) make one run a batch.
- **gpu_render3** (`ClipFrame3`): each batch's triangles are read at its offset and stride in the caller's array, framed (x, y) into the GPU's vertex buffer at that stride — kept as they are when inside every plane, else clipped in their place — so the output is each batch's own packed run (`s_pOutOff`, `s_pOutStride`); `Render2` gives each batch its attribute records at that address and stride, and draws from its vertex 0 (`gpu_render2`'s batches too, at the common stride). The Wind Waker's display no longer copies its ~7 MB of vertices a frame before the call (`gcemu's Out::prepare 5.9 -> 0.1 ms`): the machine's core, sharing the L2 and the RAM, 33.5 -> 29.9 M cycles a field.
- **gpu_render2's clipping** (`ClipFrame2`): a triangle whose three vertices are inside every plane (`V3DInsideN`: the near plane, $w > 0$, the guard band — nearly all of them) is copied once, its program's inputs only, straight into the GPU's vertex buffer (`s_verts2`, sized for the clipped fans); the others go through `V3DClipTriangleN` (every float interpolated) into the same buffer. Before, each vertex was copied four times (staged, clipped, into a buffer, into the GPU's) — ~30 MB a frame for The Wind Waker's ~100k vertices through the L2 the app cores share: its `gpu_render2` went from ~34 to ~25 ms a frame. Every 256 frames `kmsg` gets `render2`, a frame: `clip ... us`, `lists ... us`, `GPU ... us`, `target ... us`; ... vertices (the clipping and its copy; the lists, the

uniforms and the target's cache cleaning; the binning and rendering; the target after) — Wind Waker's game: ~9.8 ms, ~0.9, ~14, ~0.2 for ~80k vertices drawn.

- **Textures** (`gpu_texture`, ≤ 256 , owned by the program that made them, freed with its address space): a 32-byte `TEXTURE_SHADER_STATE` (base address, size, RGBA8, swizzle, and for UIF: *level 0 strictly UIF*, its `UB_PAD`, XOR) then the texels, in the layout the texture unit reads for that size (Mesa's `v3d_setup_slices`, `kern/v3d_tiling.h`): **LT** (a line of 4×4 -texel utiles, a side ≤ 4), **UBLINEAR** 1 / 2 columns (8×8 blocks of 2×2 utiles, width $\leq 8 / 16$), else **UIF** (columns of 4 blocks, padded rows against the page-cache banks, XOR on odd columns when aligned on them). A batch's `SAMPLER_STATE` (filter, wrap S / T) is written per frame; the fragment uniforms are `p0` = the texture state | 3 (two 16-bit words returned) and `p1` = the sampler state. Host test (every texel of every size up to 300×300 against Mesa's functions): `sh tools/tests/run_v3d_tiling_test.sh`.
 - **The QPU assembler** `tools/qpu/qpuasm` (host, `cd tools/qpu && make`): Mesa's QPU encoder / disassembler (`qpu_instr.c`, `qpu_pack.c`, `qpu_disasm.c`, MIT, vendored in `tools/qpu/mesa/`) behind a parser of the disassembler's own syntax; a line is accepted only if its encoding disassembles back to it, then each shader is checked against the instruction restrictions (Mesa's `qpu_validate.c` and its scheduler's timing rules: $r4 \geq 3$ instructions after an SFU write, $r5 \geq 2$ after `ldvary`, one TMU / SFU / VPM / TLB access a line, no regfile read right after its write, the thread-switch rules, no TLB access before the scoreboard wait). `qpuasm -d HEX...` disassembles. It re-assembles the three v52 shaders bit for bit.
 - **Straight into the target** (v52 and v53): the caller's pixel pointer is translated page by page (`AT S1E1R` → `PAR_EL1`, IRQs off); when the whole span ($(h - 1) \times stride + w$ pixels) is physically contiguous below 1 GB — a window canvas is — the tile stores (and the **KEEP** loads) go straight there: raster RGBA8 with **R/B swap** (= `0x00RRGGBB` in memory), the row pitch = the caller's stride, alpha not written (`COLOR_WRITE_MASKS` `0x8`) and cleared to 0; the span is cleaned + invalidated from the CPU caches before and after. Other buffers go through `s_Target` and a copy, as before. `kmsg` says which mode is used (when it changes).
 - **Completion by interrupt**: the V3D core interrupt (GIC SPI 74, shared by core and hub in the Pi 4's device tree; the hub's are masked) is connected at bring-up with `FLDONE` (bin done), `FRDONE` (render done) and `OUTOMEM` unmasked. The handler answers the binner's memory requests from the overflow pool (and masks `OUTOMEM` once the pool is used up) and sets the bin / render events; the drawing task sleeps on them (`WaitWithTimeout`, 2 ms slices, the frame counters `BFC` / `RFC` re-checked each time, 0.5 s at most). Without an interrupt it degrades to that 2 ms polling (logged once) and serves the binner itself.
 - **Not yet**: several frames in flight (the caller waits for its frame), mipmaps, alpha test, shaders chosen by the app.
-

Annex — useful constants

Constant	Value	File
<code>KPAGE_SIZE</code>	64 KB	<code>layout.h</code>
<code>L2_BLOCK_SIZE</code>	512 MB	<code>layout.h</code>
<code>USER_VA_BASE</code>	8 GB	<code>layout.h</code>
<code>USER_WINDOW_CANVAS</code>	12 GB	<code>layout.h</code>
<code>USER_WALLPAPER_CANVAS</code>	13 GB	<code>layout.h</code>

Constant	Value	File
KAPI_TABLE_VA	14 GB	kapi_abi.h
USER_STACK_TOP	16 GB	layout.h
USER_STACK_SIZE	1 MB	layout.h
KAPI_ABI_VERSION	68	kapi_abi.h
USER_HEAP_BASE	10 GB	layout.h
MAX_TASKS	40	sysconfig.h
ASID	8 bits (1..255; 0 = kernel)	layout.h
Kernel stack of an app task	256 KB	kernel.cpp
Screen resolution	1024×768 by default (cmdline.txt width= / height=; changed while running: screen_set, v66)	window.h / cmdline.txt
GIMAGE_TRANSPARENT	0xFF00FF	gimage.h